

DAYANANDA SAGAR UNIVERSITY

Devarakaggalahalli, Harohalli
Kanakapura Road, Bengaluru South Dt - 562112, Karnataka, India



**SCHOOL OF
ENGINEERING**

**Bachelor of Technology
in
COMPUTER SCIENCE AND ENGINEERING**

Major Project Phase-II Report

**Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence in the
Internet**

Batch: 89

By

**NU Chinnappa - ENG22CS0377
Sriram Puranik - ENG22CS0467
Tejas KM - ENG22CS0480
Kalyan Gogoi - ENG23CS1031**

Under the supervision of

**Dr. Naitik S T
Assistant Professor, CSE**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING,
SCHOOL OF ENGINEERING
DAYANANDA SAGAR UNIVERSITY,**

(2025-2026)

DAYANANDA SAGAR UNIVERSITY



**SCHOOL OF
ENGINEERING**

Department of Computer Science & Engineering
Devarakaggalahalli, Harohalli, Kanakapura Road, Bengaluru South Dt - 562112
Karnataka, India

CERTIFICATE

This is to certify that the Major Project Stage-II work titled “**Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence in the Internet**” is carried out by **Sriram Puranik (ENG22CS0467), NU Chinnappa (ENG22CS0377), Kalyan Gogoi (ENG23CS1031), Tejas Km (ENG22CS0480)**, bonafide students Eight semester of Bachelor of Technology in Computer Science and Engineering at the School of Engineering, Dayananda Sagar University, Bangalore in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering, during the year **2025-2026**.

Dr. Naitik ST

Assistant Professor
Dept. of CS&E,
School of Engineering
Dayananda Sagar University

Date:

Dr. Girisha G S

Chairman CSE
School of Engineering
Dayananda Sagar University

Date:

**Dr. Udaya Kumar Reddy
K R**

Dean
School of Engineering
Dayananda Sagar
University

Date:

Name of the Examiner

Signature of Examiner

- 1.
- 2.
- 3.

DECLARATION

We, **Sriram Puranik (ENG22CS0467), NU Chinnappa (ENG22CS0377), Kalyan Gogoi (ENG23CS1031), Tejas Km (ENG22CS0480)**, are students of the Eight semester B.Tech in **Computer Science and Engineering**, at School of Engineering, **Dayananda Sagar University**, hereby declare that the Major Project titled “**Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence in the Internet**” has been carried out by us and submitted in partial fulfilment for the award of degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025-2026**.

Student

Signature

Name1: Sriram Puranik

USN : ENG22CS0467

Name2: Tejas KM

USN : ENG22CS0480

Name3: NU Chinnappa

USN : ENG22CS0377

Name4: Kalyan Gogoi

USN : ENG23CS1031

Place : Bangalore

Date :

ACKNOWLEDGEMENT

First, we take this opportunity to express our sincere gratitude to School of Engineering & Technology, Dayananda Sagar University for providing us with a great opportunity to pursue our Bachelor's degree in this institution.

We would like to thank **Dr. Udaya Kumar Reddy K R, Dean, School of Engineering, Dayananda Sagar University** for his constant encouragement and expert advice.

It is a matter of immense pleasure to express our sincere thanks to **Dr. Girisha G S, Department Chairman, Computer Science and Engineering, Dayananda Sagar University**, for providing right academic guidance that made our task possible.

We would like to thank our guide **Dr. Naitik ST, Assistant Professor, Dept. of Computer Science and Engineering, Dayananda Sagar University**, for sparing his/her valuable time to extend help in every step of our project work, which paved the way for smooth progress and fruitful culmination of the project.

We would like to thank our **Project Coordinators Dr. Sivananda Lahari Reddy, Dr. Kumar Dilip and Dr. Naresh P** as well as all the staff members of Computer Science and Engineering for their support.

We are also grateful to our family and friends who provided us with every requirement throughout the course.

We would like to thank one and all who directly or indirectly helped us in the Project work.

.

TABLE OF CONTENTS

LIST OF FIGURES.....	35
ABSTRACT	8
CHAPTER 1 INTRODUCTION.....	9
CHAPTER 2 PROBLEM DEFINITION.....	10
CHAPTER 3 LITERATURE SURVEY	11-13
CHAPTER 4 PROJECT DESCRIPTION	14-17
CHAPTER 5 REQUIREMENTS	17-22
CHAPTER 6 METHDOLOGY.....	23-32
CHAPTER 7 EXPERIMENTATION	33-42
CHAPTER 8 RESULT	43-62
CHAPTER 9 DELIVERABLES	59-63
REFERENCES.....	64-65

NOMENCLATURE USED

Acronym/Term	Full Form / Definition
IS-HAITI	Intellectual Smart Hybrid AI System for Real-Time Threat Intelligence in the Internet
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
SAGE	GraphSAGE (Inductive representation learning on large graphs)
SMOTE	Synthetic Minority Over-sampling Technique
X-TIS	Explainability & Transparency Intelligence System
CADA	Continuous Adaptation & Defense Architecture
TCN	Temporal Convolutional Network
C2	Command & Control
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
SOC	Security Operations Center
APT	Advanced Persistent Threat
XAI	Explainable Artificial Intelligence
DLP	Data Loss Prevention
EDR	Endpoint Detection & Response
NSL-KDD	NSL-KDD Network Intrusion Detection Dataset
CICIDS	Canadian Institute for Cybersecurity Intrusion Detection System Dataset
PCAP	Packet Capture file format
CSV	Comma-Separated Values
ROC-AUC	Receiver Operating Characteristic - Area Under the Curve
F1-Score	Harmonic mean of Precision and Recall
MFA	Multi-Factor Authentication
DNS	Domain Name System

LIST OF FIGURES

Fig. No.	Description of the figure	Page No.
1	Fig 1: System Architecture	14
2	Fig 2: High-Level Workflow of the IS-HAITI Threat Detection System	21
3	Fig 8.1: Service-Protocol Interaction Graph (service_protocol_graph.png)	31
4	Fig 8.2: GNN Threat Score Dashboard (gnn_threat_dashboard.png) & Pooled ROC Curves (Our-GNN vs Baselines, 5-fold CV)	32
5	Fig 8.3: Top Services by Predicted Attack Probability & Fig 8.4: GNN Threat Score Histogram (gnn_threat_score_hist.png)	33
6	Fig 8.5: NetBIOS/SMB-Related Subgraph & Fig 8.6: IRC Subgraph - C2 Coordination Pattern (IRC_subgraph.png)	34
7	Fig 8.6: PM_Dump Subgraph & Attribution View; Fig 8.7: IRC SHAP Feature Attribution (annotated_shap_IRC.png); Fig 8.8: NetBIOS-SSN Subgraph (netbios_ssn_subgraph.png); Fig 8.10: NetBIOS-DGM Subgraph; Fig 8.11: PM_Dump Subgraph; Fig 8.13: DNS/Name Subgraph	35
8	Fig 8.12: PM_Dump SHAP Feature Attribution & Fig 8.14: DNS SHAP Feature Attribution (annotated_shap_name.png)	36-37
9	Fig 8.13: Website Entry Point; Fig 8.14: Consolidated Metrics; Fig 8.15: Metrics One-Pager Summary (metrics_one_pager.png); Fig 8.16: Transformer Combined Risk Histogram; Fig 8.17: Autoencoder Anomaly Score Histogram; Fig 8.18: SMS Spam Collection Text Threat Analysis Report	38
10	Fig 8.19: URL Threat Analysis Report - Category B; Fig 8.20: Network C/D Category HTML Report; Fig 8.21: E, F, G, H Detection Pages	39
11	Fig 8.22: Category E - Botnets/C2; Fig 8.23: Category F - Data Exfiltration; Fig 8.24: Category G - Malware Behavioral; Fig 8.25: Category H - Multi-Stage/Unknown Subtype Counts	40-41

ABSTRACT

Most intrusion detection systems today work at the level of individual network connections. They look at a single packet or flow and decide whether it looks malicious. This approach has a known weakness: many real attacks are spread across multiple connections and multiple services, and no single connection looks suspicious on its own.

IS-HAITI is a hybrid AI-based threat detection and intelligence system that covers all of these threat types under one platform. It combines a Transformer model for network flow analysis, a Graph Neural Network for detecting coordinated attack behavior across services, a rule-based engine for known patterns, a content classifier for spam and phishing text, a URL analyzer for malicious links, and a deepfake detector for synthetic media. Only the models relevant to the input type activate, and every detection is explained by X-TIS using SHAP and LIME telling the user exactly which signals triggered the alert, not just that something was flagged.

The framework integrates learned model outputs with rule-supported analysis and produces structured detection reports together with explainability outputs through the Explainable Threat Intelligence System (X-TIS). The system demonstrates capability in identifying phishing and spam communications, malicious URLs and credential harvesting websites, network reconnaissance activities, brute-force authentication attempts, service enumeration patterns, and botnet-related communication behavior.

CHAPTER 1 INTRODUCTION

Cyber attacks scans the network to see what is available, Then they move around inside the network. Then they quietly take data out If you look at any one of these steps on its own, it might not look suspicious. It is only when you look at all the steps together, across different services, that the attack pattern becomes clear. Most intrusion detection systems miss this because they examine each network connection in isolation.

The increasing complexity of cyber-attacks has created significant challenges for traditional network security systems. Modern attackers rarely rely on a single exploit instead, attacks typically involve multiple stages such as credential attacks, malicious communication establishment, and persistence mechanisms. These activities often appear normal when analyzed independently but reveal malicious intent when behavioral and structural relationships are considered together.

IS-HAITI addresses both of these problems. For network traffic, it uses two layers of analysis. The first layer is a Transformer model that scores each individual connection. The second layer is a Graph Neural Network (GNN) that looks at how services in the network communicate with each other as a group, which is where multi-stage attack patterns become visible. For non-network threats, it has separate modules for URL analysis, text message analysis, and deepfake detection. All of these sit behind a rule-based verification engine and an explainability system called X-TIS that tells the user exactly why something was flagged.

Our project framework focuses on hybrid cyber threat analysis by combining behavioral learning models. The system learns attack behavior from multiple intrusion detection datasets and evaluates how risk propagates across interacting services. Instead of replacing traditional approaches, the framework integrates learning-based intelligence with interpretable analysis outputs.

CHAPTER 2 PROBLEM DEFINITION

Our project addresses a major limitation in current threat detection systems, which typically analyze data in isolation, rely on a single model, and fail to provide meaningful explanations. Existing machine learning-based intrusion detection systems evaluate each network connection individually, making it difficult to identify complex attacks like port scans or lateral movement that emerge only when multiple activities are viewed together. Additionally, most systems focus solely on network traffic, ignoring other critical threat sources such as phishing URLs, malicious websites, or deepfake media. Another key issue is the lack of explainability when a threat is detected, users are often given only a simple label without any insight into why the decision was made, reducing trust and limiting further investigation. Furthermore, these models are usually static and do not adapt to evolving attack patterns, leading to decreased effectiveness over time. Finally, many existing solutions lack user-friendly interfaces, requiring technical expertise to operate. To overcome these challenges, this project proposes an integrated platform that performs multi-level analysis, supports multiple input types, provides clear explanations for detections, adapts to changing threats, and is accessible through web, browser, and mobile interfaces.

CHAPTER 3 LITERATURE REVIEW

- **3.1 Transformer Models on Tabular Data**

The Transformer architecture was introduced by Vaswani et al. in 2017 for language translation [1]. The core idea is self-attention: the model learns which features to focus on when producing each output, rather than processing all features equally. This turns out to work well on tabular data too. Gorishniy et al. (2021) showed that when each feature is turned into an embedding (a technique called feature tokenization) and those embeddings are passed through Transformer encoder layers, the model can match or outperform gradient-boosted trees on many tabular benchmarks [2]. Network intrusion datasets like NSL-KDD and CICIDS2017 are tabular each row is a flow with numerical and categorical features which is why a Transformer is a reasonable choice for the flow-level detection model in IS-HAITI.

- **3.2 Graph Neural Networks for Network Security**

A Graph Neural Network is a model that operates on graph-structured data. Nodes hold features, edges represent relationships, and the model passes information between neighboring nodes to update each node's representation. Kipf and Welling (2017) introduced Graph Convolutional Networks (GCN), where each node aggregates a weighted average of its neighbors' features [3]. Velickovic et al. (2018) introduced Graph Attention Networks (GAT), which learn how much weight to give each neighbor rather than treating all neighbors equally [4]. Hamilton et al. (2017) proposed GraphSAGE, which samples a fixed number of neighbors at each layer, making the approach practical on large graphs [5].

In network security, the natural way to build a graph is to treat services as nodes and communication between services as edges. If service A communicates with service B, there is an edge between them. When the GNN processes this graph, each service node's score is influenced by its neighbors. So if several services around a node are showing attack-linked behavior, the node's risk score rises even if its own traffic looks normal on its own. This is the core advantage of the GNN in IS-HAITI it captures the coordinated behavior that single-flow classifiers miss.

- **3.3 Rule-Based Detection**

Rule-based detection is the oldest approach in intrusion detection. Systems like Snort use manually written rules that describe known attack patterns – for example, a SYN flood is detected when more than a threshold number of SYN packets arrive from different IPs within a short time window. Rule-based systems are fast, transparent, and reliable for known patterns. Their weakness is that they cannot detect anything they were not written for [6].

In IS-HAITI, the rule engine is not the primary detector. It works alongside the ML models. When both the GNN and a rule fire on the same service, the alert confidence is high. When only the rule fires, the alert is medium confidence. When only the GNN flags something, it is lower confidence but still reported. This layered approach uses the strengths of both without being limited by either.

- **3.4 Explainable AI: SHAP and LIME**

SHAP (SHapley Additive exPlanations) was proposed by Lundberg and Lee (2017) [7]. It computes how much each feature contributed to a specific prediction. The contribution values are based on Shapley values from game theory – the idea is to measure how much the prediction changes when a feature is included versus excluded, averaged across all possible orderings of features. The result is a signed score for each feature: positive means it pushed the prediction toward attack, negative means it pushed away from attack.

LIME, proposed by Ribeiro et al. (2016), works differently [8]. It creates many slightly modified versions of the input, runs the model on all of them, and fits a simple linear model to the pattern of outputs. The coefficients of that linear model serve as the explanation. LIME is particularly useful for text inputs because it can show which words in a message drove the classification.

In IS-HAITI, SHAP is used for the Transformer and GNN explanations because it gives theoretically consistent attributions across the full feature set. LIME is used for the content classifier because it works well with text. X-TIS (the explainability layer) calls both and formats the output as a plain-language reason that tells the user which signals triggered the detection.

- **3.5 Threat Detection**

Phishing and spam detection has been studied extensively. Earlier approaches used simple bag-of-words features with naive Bayes classifiers. More recent work uses TF-IDF combined with logistic regression or SVMs, which still performs competitively on short-text classification tasks like SMS spam detection. The SMS Spam Collection dataset, introduced by Almeida et al. (2011), is a standard benchmark for this task with 5,574 labeled messages [9]. For URL-based threat detection, lexical features—things like URL length, the number of dots, whether the hostname is an IP address, whether the URL contains login-related keywords—have been shown to be effective discriminators between phishing and benign URLs without needing to actually visit the page [10].

- **3.6 Deepfake Detection**

Deepfakes are media files—images, video, or audio—that have been synthetically generated or manipulated, usually to make it look like someone said or did something they did not. The FaceForensics++ benchmark, introduced by Rossler et al. (2019), provides a large collection of real and manipulated videos for training and evaluating detectors [11]. Research has shown that current deepfake generation methods leave detectable patterns in the frequency domain of images—specifically, upsampling artifacts that appear at regular intervals in the Fourier spectrum. For audio deepfakes, classifiers trained on mel-frequency cepstral coefficients (MFCCs) can distinguish natural speech from neural text-to-speech output because synthetic voices have different formant transition patterns [12].

- **3.7 Concept Drift and Online Learning**

Concept drift happens when the statistical distribution of the data a model sees in deployment shifts away from the distribution it was trained on. In intrusion detection, this is expected: new attack tools change traffic patterns, network configurations change, software is updated. Gama et al. (2014) surveyed drift detection methods and highlighted ADWIN (ADaptive WINdowing) as a reliable statistical test for detecting changes in a data stream [13]. ADWIN maintains a sliding window over incoming data and shrinks it when a statistically significant change is detected. IS-HAITI uses ADWIN on incoming flow features. When drift is detected, the system triggers an incremental model update using online learning so it can adapt without a full retrain from scratch.

CHAPTER 4 PROJECT DESCRIPTION

The central idea of IS-HAITI is this: a user submits something a URL, a message, a file, a PCAP, or live traffic and the system figures out what kind of input it is, activates the relevant models, combines their results, and tells the user what threat was found.

4.1 System Architecture

The system has five layers:

1. **Input Layer**
2. **Preprocessing Layer**
3. **Detection Layer**
4. **Interface Layer**

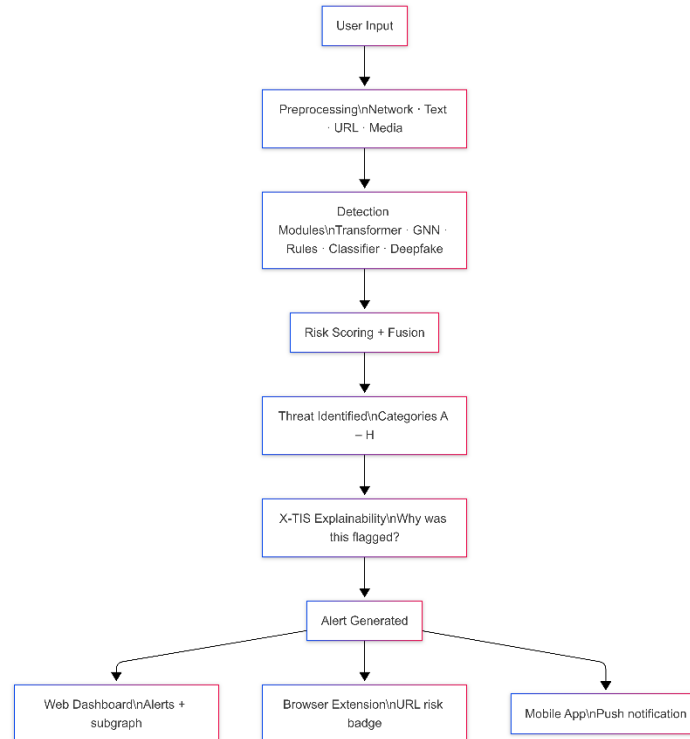


Fig 1. System Architecture

4.1.1 Input Layer

The entry point of the system. Users interact through one of three interfaces: the web platform for full dashboard access, the browser extension for real-time URL risk assessment while browsing, and the mobile application for on-the-go monitoring. Each interface forwards the input to the preprocessing layer after identifying its type.

4.1.2 Preprocessing Layer

Raw input is cleaned and transformed into a structured format suitable for detection. Network traffic captured as PCAP files is converted into 77-feature flow records using CICFlowMeter, followed by normalization and missing value handling. Text inputs are vectorized using TF-IDF. URLs undergo lexical feature extraction. Media inputs such as video, images, and audio are processed through frame extraction, face detection, and spectrogram generation respectively. The preprocessing layer ensures that every input arrives at the detection layer in a consistent, model-ready format.

4.2.3 Detection Layer

The core intelligence of the system. Five specialized modules sit in this layer, each targeting a different threat surface:

- The **Transformer model** analyzes individual network flows and assigns an attack probability score to each connection based on behavioral patterns learned from large-scale intrusion datasets.
- The **GNN ensemble** constructs a service-level graph from network traffic and identifies coordinated attack behavior patterns that only become visible when multiple services are observed together. This is the layer that catches multi-stage attacks that no single-flow model can detect.
- The **Rule-based engine** fires on known attack signatures and threshold violations, providing fast and reliable coverage for well-documented threat patterns.
- The **Content classifier** analyzes text inputs emails, SMS messages, and chat content to detect spam, phishing, scams, and social engineering attempts.

- The **Deepfake detector** identifies synthetic or manipulated media, flagging AI-generated video, images, and audio that may be used in deception campaigns.

For network traffic, the Transformer, GNN, and rule engine all run simultaneously. Their outputs are combined through a risk fusion step before passing to the next layer. For all other input types, only the relevant module activates.

4.2.4 Explainability Layer - X-TIS

Every detection, regardless of input type or module, passes through X-TIS (Explainability and Transparency Intelligence System) before an alert is raised. X-TIS uses SHAP values to explain network model detections, LIME to explain content classifier decisions, and frequency-domain attribution to explain deepfake detections. The output is a plain-language statement telling the user not just that a threat was found, but which specific signals caused the system to flag it. This makes the system usable by security analysts who need to investigate and act on alerts, not just receive them.

4.2.5 Interface Layer

The final layer delivers results to the user through the same three interfaces that accepted the input. The web dashboard presents the full picture – alerts, risk scores, X-TIS explanations, and for network threats, a visual subgraph showing which services are involved. The browser extension overlays a risk badge directly onto URLs as the user browses. The mobile application sends push notifications for high-severity alerts so that critical threats are never missed.

CHAPTER 5 REQUIREMENTS

The IS-HAITI system is implemented as a hybrid cyber threat intelligence platform consisting of multiple analytical components including graph-based service risk detection, Transformer-based behavioral learning, fusion modeling, content threat detection, malicious URL analysis, and explainable intelligence generation.

5.1 Functional Requirements

Functional requirements describe the operations that the system must perform to achieve cyber threat analysis.

5.1.1 Input Handling

The system accepts five types of input: a URL pasted by the user, a text message or email body, an image, audio, or video file for deepfake analysis, a PCAP file for batch network analysis, and live network traffic captured via T-Shark or the browser extension. Each input type is routed to the correct module automatically.

5.1.2 Network Traffic Processing

For PCAP inputs, the system calls CICFlowMeter to generate 77-feature bidirectional flow records. For live traffic, T-Shark captures packets on a configurable interface and writes them to rolling PCAP files that CICFlowMeter processes. The resulting flow CSV feeds into the preprocessing step without manual intervention.

5.1.3 Transformer Model

The Transformer model is trained on NSL-KDD, CICIDS2017, and UNSW-NB15. During training it saves checkpoints and logs per-epoch metrics. During inference it takes a preprocessed flow CSV and outputs a per-flow attack probability score. These scores are exported to CSV and passed downstream to the GNN.

5.1.4 Graph Neural Network

The GNN takes the service interaction graph built from aggregated Transformer scores and traffic statistics and outputs a per-service attack probability. It runs as an ensemble of three architectures (GCN, GAT, GraphSAGE). The output is a risk-ranked list of services and subgraph visualizations of the highest-risk clusters.

5.1.5 Rule-Based Engine

The rule engine runs on flow data alongside the ML models. It checks for: SYN flood patterns (high SYN ratio, small packet sizes), distributed brute force (high connection frequency from many distinct IPs in a short window), non-standard port usage for IRC-like services, and volume anomalies for potential exfiltration. When a rule fires, it raises the alert severity of any ML detection on the same service.

5.1.6 X-TIS Explainability

X-TIS runs after every detection. For Transformer and GNN outputs, it uses SHAP to compute which features contributed most to the prediction and in which direction. For content classifier outputs, it uses LIME to identify which words or phrases drove the classification. For deepfake outputs, it identifies which frequency-domain or spectral features triggered the alert. The explanation is always presented in plain language alongside the risk score.

5.1.7 Content and URL Detection

The content classifier uses TF-IDF with unigram and bigram features on SMS and email text

to detect spam, phishing, scam, and BEC. The URL module extracts 22 lexical features from each link including length, subdomain count, IP-as-hostname flag, URL shortener detection, entropy of path components, presence of brand keywords, and login-related keywords and scores each URL with a gradient-boosted classifier.

5.1.8 Deepfake Detection

For images: face regions are detected, a 2D DFT is applied, and a CNN trained on FaceForensics++ classifies based on frequency-domain patterns. For audio: MFCCs (40 coefficients) and LFCCs (40 coefficients) are extracted and an LSTM classifier identifies synthetic speech. For video: the image detector runs on sampled frames and temporal consistency between frames is also checked.

5.1.9 Continuous Learning and Drift Detection

ADWIN monitors incoming flow features for distribution changes. When drift is detected across multiple features, an incremental update is triggered: the Transformer classification head is updated using SGD with a reduced learning rate on the most recent data, combined with a replay buffer of historical samples to prevent forgetting. Full retraining is triggered when cumulative drift exceeds a threshold.

5.2 Non-Functional Requirements

5.2.1 Performance Requirements

- **Scalability:** The system architecture should support processing of larger intrusion detection datasets and additional threat intelligence sources without major redesign. The modular pipeline must allow integration of new datasets, models, and detection modules in future extensions.

- **Reliability:** The system should generate consistent detection results when executed multiple times on similar input data. Model training, inference, and fusion processes must maintain stable performance across different datasets
- **Usability:** Detection outputs should be easily interpretable through structured CSV/JSON reports and human-readable explainability summaries (X-TIS reports), enabling understanding even for users without deep machine learning expertise.
- **Performance:** The system should provide reasonable training and inference time using GPU acceleration and optimized preprocessing pipelines, ensuring efficient execution during experimentation and testing.
- **Maintainability:** The codebase should follow modular design principles, allowing independent modification of preprocessing, Transformer training, GNN fusion, detection modules, and explainability components without affecting the entire system.
- **Security:** The system should follow safe data handling practices, ensuring datasets are processed locally without unauthorized exposure, and all model outputs are generated within controlled experimental environments.

5.3 Software Requirements

- **Operating System:** Windows 10 / Linux / macOS
- **Programming Language:** Python (Version 3.x)
- **AI/Machine Learning Frameworks:**
 - PyTorch
 - TensorFlow (optional)
- **Supporting Libraries:**
 - Scikit-learn
 - NumPy
 - Pandas
 - NetworkX / PyTorch Geometric (for graph modelling)
- **Visualization Tools:**
 - Matplotlib
 - Seaborn
- **Development Environments:**
 - Jupyter Notebook
 - VS Code or any Python IDE

5.4 Hardware Requirements

- A computer system with a minimum of **Intel Core i5 processor or equivalent**.
- **8 GB RAM** or higher for smooth execution of machine learning models.
- At least **500 GB of storage** for storing datasets, trained models, and project files.
- A **GPU** (such as NVIDIA GPU with CUDA support) is recommended for faster model training, though the system can also run on CPU for basic experiments.
- Stable **internet connection** for accessing datasets, libraries, and research resources.

CHAPTER 6 METHDOLOGY

This chapter explains how each part of the IS-HAITI pipeline actually works from receiving input to generating an explained alert. The workflow shown in Figure 2 below covers every stage: the three input interfaces, the input type router, four preprocessing paths, five detection modules, output fusion and risk scoring, eight threat categories, X-TIS explainability, and the three alert delivery channels.

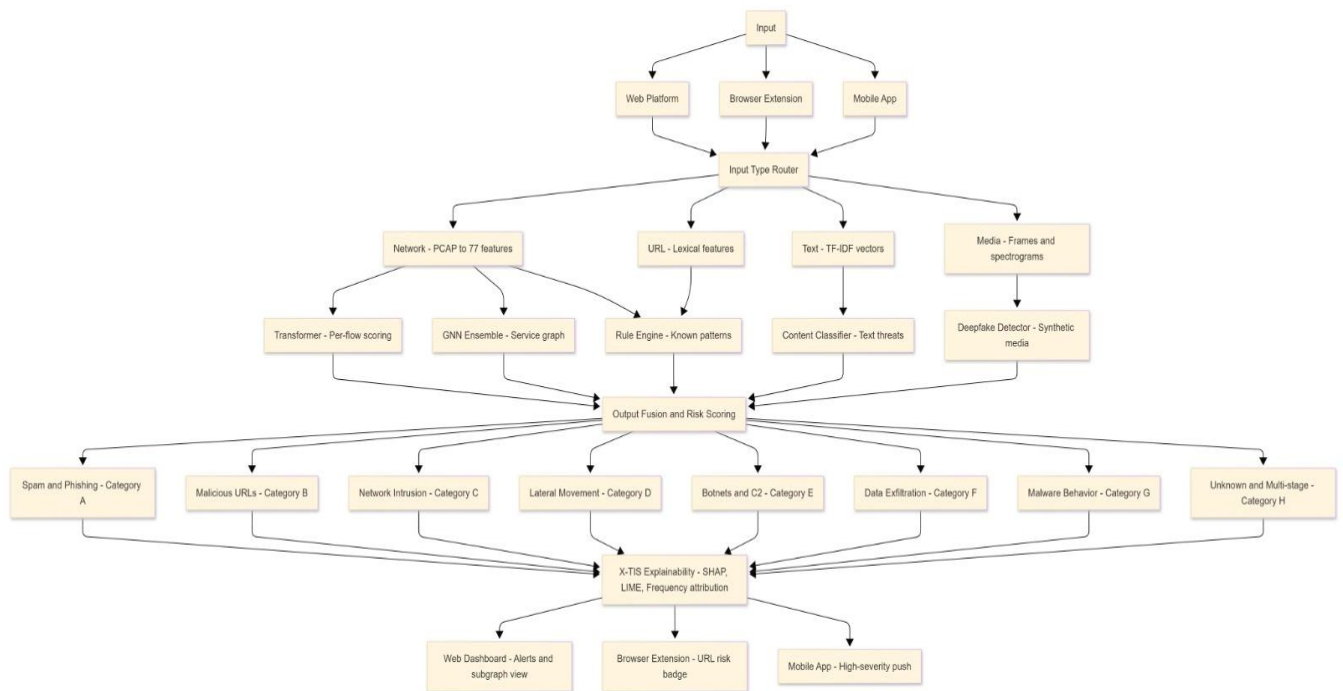


Fig 2: High-Level Workflow of the IS-HAITI Threat Detection System

Figure 1: illustrates the overall working flow of the proposed IS-HAITI threat detection system. The process begins when a user uploads network traffic data or log files into the system. This data is first passed through a preprocessing stage where it is cleaned, normalized, and converted into a suitable format for analysis.

6.1 Input and Interfaces

As shown in **Figure 2**, the system receives input through three interfaces: the Web Platform, the Browser Extension, and the Mobile App. Each interface is designed for a different use case but all three connect to the same backend pipeline.

The Web Platform is the main interface. It lets a user submit any type of input a URL, a text message or email body, a media file such as an image, audio, or video, or a PCAP file for network analysis. Results are displayed on screen with the full alert details, risk score, and X-TIS explanation. For network threats, a visual subgraph of the affected services is also shown. The platform keeps a running alert history so security teams can review past detections and update alert status.

The Browser Extension runs passively as the user browses. Every time a page loads, the URL is checked by the detection backend and a risk badge appears on the browser toolbar. If the URL is flagged, a tooltip shows the X-TIS reason directly in the browser without the user needing to open the dashboard. The Mobile App connects to the backend through a secure API. Its main function is to send push notifications when High severity alerts are generated, so analysts are notified even when they are away from the dashboard. From the app, the analyst can read the full alert detail and mark it as reviewed, escalated, or dismissed.

6.2 Input Type Router

Once input arrives from any of the three interfaces, it passes through the Input Type Router. This step identifies what type of input was submitted and decides which preprocessing path and detection modules should be activated.

The router checks the MIME type of any uploaded file, the structure of text input to tell apart plain text from a URL, and which interface the submission came from. Based on this, the input is classified into one of four types: Network (PCAP files or live traffic captured via T-Shark), URL, Text, or Media. Only the modules relevant to that type are activated. A PCAP file does not trigger the content classifier or deepfake detector. A text message does not go through CICFlowMeter or the GNN. This routing is fully automatic the user does not select a module manually.

6.3 Preprocessing

Before any detection module runs, raw input is converted into a form that the models can process. As shown in Figure 2, the four preprocessing paths are: Network PCAP to 77 features, URL Lexical features, Text TF-IDF vectors, and Media Frames and spectrograms.

Network Traffic: For PCAP input, CICFlowMeter parses the file and produces one row per bidirectional flow. Each row has 77 features: forward and backward packet length statistics (mean, max, min, standard deviation), inter-arrival time statistics in both directions, TCP flag counts, flow duration, subflow byte and packet averages, and active/idle time statistics. For live traffic, T-Shark captures packets on a configurable interface and writes them to rolling PCAP files that CICFlowMeter processes.

Raw CICFlowMeter output has two known problems. When flow duration is zero, rate feature calculations produce infinite values. Incomplete flows produce missing values. Infinite values are replaced with the per-feature training maximum and missing values are filled with training medians. Categorical fields like protocol and service name are integer-encoded. All numerical features are then Min-Max normalized using parameters fitted on the training data only, so no test or live data information leaks into normalization. Using the same CICFlowMeter tool for both training preparation and live inference is a deliberate design choice it ensures the model always sees feature values computed the same way.

URL Input: 22 lexical features are extracted directly from the URL string without visiting the page. These include URL length, number of subdomains, whether the hostname is an IP address, whether a URL shortener is in use, entropy of the path components, presence of brand-name

keywords that could indicate impersonation, presence of login-related keywords such as 'verify' or 'account', count of special characters, and top-level domain type.

Text Input: SMS messages and email bodies are lowercased, stop words are removed, and TF-IDF vectorization is applied with unigrams and bigrams. If the message contains URLs, those are extracted and sent through the URL preprocessing path as well.

Media Input: For images, a face detector identifies face regions first. For video, frames are sampled at regular intervals and the image-level detector is applied to each sampled frame. For audio, 40 MFCC coefficients and 40 LFCC coefficients are extracted from 25 ms frames with a 10 ms hop between frames. These coefficients capture the spectral shape of each audio frame, which reflects the vocal tract characteristics of the speaker and differs clearly between real and synthetic speech.

6.4 Transformer Model Per-Flow Scoring

The Transformer model is the first detection layer for network traffic. It works at the individual flow level and assigns an attack probability to each connection. The architecture follows a feature tokenization design: each of the 77 flow features is projected to a 64-dimensional embedding using a learned linear layer, converting a flat row of numbers into a sequence of 77 vectors. These 77 vectors are passed through 4 Transformer encoder layers with 8 attention heads each.

The attention mechanism allows the model to learn which combinations of features matter for a prediction. For example, it can learn on its own that high SYN count combined with small packet size is a strong indicator of a port scan, without being given that rule explicitly. After the encoder layers, a classification head of two fully connected layers with ReLU activation and 0.3 dropout produces a single output value. A sigmoid converts this to a probability between 0 and 1. This is the per-flow attack probability that is passed into the service graph construction step. Training uses Binary Cross-Entropy loss with class weights that are the inverse of class frequency, to handle the imbalance between benign and attack flows in intrusion detection datasets. The optimizer is Adam with learning rate 1e-3, weight decay 5e-5, batch size 4096,

and gradient clipping at norm 1.0. Training runs for up to 100 epochs with early stopping at patience 15 based on validation loss. The checkpoint with the best validation ROC-AUC is kept for deployment. The model is trained on NSL-KDD, CICIDS2017, and UNSW-NB15.

6.5 Service Graph Construction and GNN Ensemble

After the Transformer scores each flow, those scores are aggregated per service. A service is identified by mapping the destination port of each flow to a known service name. For each service, the system computes: mean and maximum attack probability across all flows to that service, standard deviation of attack probabilities, total flow count, mean and standard deviation of source and destination byte counts, TCP/UDP/ICMP protocol ratios, count of distinct source IPs, and burst rate in flows per second. These values form the node feature vector for that service in the graph.

An edge is drawn between two services whenever direct communication between them appears in the flow data. The edge weight equals the number of flows between that pair. The graph is stored as a PyTorch Geometric Data object with the node feature matrix, edge index, edge weights, and node labels from the dataset ground truth. For NSL-KDD, this process produces 70 service nodes.

Three GNN architectures are trained independently on this service graph:

- **GCN (Graph Convolutional Network):** each node aggregates a normalized weighted sum of its neighbour's features at each layer. Three convolution layers allow information to travel up to three hops through the graph. GCN is good at picking up local structural patterns around each service.
- **GAT (Graph Attention Network):** similar to GCN but the aggregation weights are learned through an attention mechanism. This lets the model figure out which neighbouring services carry more useful information for each node being classified.

- **GraphSAGE:** samples a fixed number of neighbours at each layer instead of using the full neighbourhood. This makes it scalable to larger graphs and the sampling introduces regularization.

All three use hidden dimension 64, 3 message-passing layers, dropout 0.3, and learning rate $1e-3$. Each model produces a per-node attack probability. The final score for each service is a weighted average of the three outputs, with weights tuned on the validation set to maximize F1-score. Using an ensemble consistently outperforms any single GNN architecture.

This GNN layer is what allows IS-HAITI to detect multi-stage and coordinated attacks. If several services around a node are showing attack-linked behavior, that node's risk score rises even if its own individual traffic looks normal. This cross-service pattern is exactly what port scanning, lateral movement, and botnet activity look like and it is invisible to any model that only looks at one flow at a time.

6.6 Rule-Based Engine Known Patterns

The rule engine runs at the same time as the Transformer and GNN on the same network flow data. It applies four checks:

- **Port scan / SYN flood:** more than 95% of flows to a service have the SYN flag set and mean packet size is below 100 bytes.
- **Distributed brute force:** more than 500 flows to a service from more than 50 distinct source IPs within one minute.
- **C2 indicator:** a service identified as IRC-like by name or by port number (known IRC ports 6660–6669) is receiving connections on non-standard ports.
- **Exfiltration indicator:** total destination bytes for a service exceed 5 standard deviations above its historical mean in the current measurement window.

When a rule fires on a service that the GNN also flagged, the alert severity is raised to High. When only a rule fires, severity is Medium. When only the GNN flags something, severity is Low or Medium depending on the probability score. This three-level scheme tells the analyst exactly how much confidence to place in each alert. The rule engine handles known patterns reliably while the GNN catches newer and more complex behavior together they cover more ground than either can alone.

6.7 Content Classifier and URL Analyzer

When input is a text message or email body, the content classifier runs. It uses TF-IDF with unigram and bigram features to represent the text. The classifier is a gradient-boosted model trained to detect spam, phishing, scam messages, and Business Email Compromise (BEC). Training data includes the SMS Spam Collection dataset with 5,574 labeled messages and labeled email datasets. If the message contains URLs, those are automatically extracted and passed to the URL analyzer as well.

The URL analyzer takes the 22 lexical features extracted during preprocessing and scores each URL using a separate gradient-boosted classifier. It identifies malicious links and credential harvesting sites based on the structure of the URL alone, without visiting the page. This makes the module both safe and fast. Features like IP-as-hostname, login keywords in the path, brand keywords that suggest impersonation, URL shortener detection, and suspicious TLD type are the main signals the classifier uses.

Both classifiers output a probability between 0 and 1. Anything above the detection threshold passes to X-TIS for explanation and is turned into an alert. When a message contains a URL, the combined signal from both the text classifier and the URL analyzer is used – a suspicious message containing an also-suspicious URL raises the overall confidence of the detection.

6.8 Deepfake Detection Pipeline

When an image is submitted, face regions are detected first. Each detected face is transformed into the frequency domain using a 2D Discrete Fourier Transform (2D DFT). GAN-generated images have characteristic upsampling artifacts – regular periodic patterns in the high-frequency part of the spectrum that photographs taken by a real camera do not have. A ResNet-based CNN trained on the FaceForensics++ benchmark classifies each face region as real or synthetic based on these spectral patterns.

For video input, the same image-level detector runs on frames sampled at regular intervals. A temporal consistency check is also applied across consecutive frames. Face-swap deepfakes

often produce small but detectable inconsistencies in how the face moves from frame to frame the temporal check picks up this pattern even when individual frames are borderline.

For audio input, the 40 MFCC and 40 LFCC coefficients extracted during preprocessing are fed into an LSTM classifier trained on real and synthetic speech samples. Synthetic voices produced by neural text-to-speech systems have different formant transition patterns from natural speech. The LSTM processes the full sequence of coefficient frames and outputs a probability that the recording is synthetically generated. X-TIS then reports which frames and which coefficients deviated most from natural speech patterns.

6.9 Output Fusion, Risk Scoring, and Threat Categories

As shown in **Figure 2**, all module outputs feed into a single Output Fusion and Risk Scoring step. For network traffic, the Transformer per-flow scores, GNN per-service scores, and rule engine signals are combined here using the three-level severity scheme described in Section 6.6. For all other input types, the single module's probability score becomes the risk score directly.

After fusion, every detected threat is assigned to one of eight categories. These eight categories cover every type of threat the system can detect across all input types:

- **Category A Spam and Phishing:** spam messages, phishing emails, scam content, and Business Email Compromise detected by the content classifier.
- **Category B Malicious URLs:** phishing websites, fake login pages, credential harvesting sites, and brand impersonation links detected by the URL analyzer.
- **Category C Network Intrusion:** port scanning, service enumeration, and reconnaissance activity detected by the Transformer and rule engine.
- **Category D Lateral Movement:** SMB-based lateral movement, internal pivoting, and NetBIOS attack patterns detected by the GNN through cluster behavior in the service graph.
- **Category E Botnets and C2:** IRC-based botnet behavior, HTTP/HTTPS beaconing, and DNS tunneling detected by the Transformer and GNN on IRC and DNS service nodes.
- **Category F Data Exfiltration:** abnormal outbound data volume detected by the Transformer and the exfiltration rule.

- **Category G Malware Behavior:** backdoor communication, process memory dump indicators, and persistence-linked traffic detected through GNN subgraph analysis.
- **Category H Unknown and Multi-Stage Attacks:** attacks that span multiple services over time and do not match a single signature, detected through structural anomalies in the GNN service graph and flagged for analyst review.

6.10 X-TIS Explainability, SHAP, LIME, and Frequency Attribution

Every detection, from every module, passes through X-TIS Explainability and Transparency Intelligence System before an alert is sent. X-TIS answers one question for every detection: why was this flagged? Every alert the system delivers includes a specific readable reason, not just a threat label and a score. An analyst who gets an unexplained output cannot verify it or act on it confidently. X-TIS makes sure that never happens.

For Transformer detections: SHAP DeepExplainer computes the contribution of each of the 77 flow features to that prediction. The top 5 features by absolute SHAP value are shown. A positive value means the feature pushed the score toward attack; a negative value means it pushed away. Example output: 'Flagged because: high SYN count (↑), large source bytes (↑), low destination bytes (↓), high flag count (↑), short flow duration (↑).'

For GNN detections: gradient-based attribution computes how much each node feature of the flagged service influenced its classification. The 2-hop subgraph around that service the service itself, its direct neighbours, and their neighbours is extracted and shown as a graph diagram. Nodes are coloured by risk: green for low, orange for medium, red for high. The analyst can see exactly which cluster of services is behaving suspiciously together.

For rule-based detections: the explanation is the rule that fired and the actual values that triggered it. For example: 'Rule triggered: SYN ratio 0.97, mean packet size 48 bytes matches port scan pattern.' No additional attribution is needed the rule itself is the explanation.

For content classifier detections: LIME builds a local linear model around the specific message. It generates many modified versions of the message dropping or replacing words runs the classifier on all of them, and fits a linear model to the pattern of outputs. The words that contributed most to the classification are shown. Example: 'Flagged because: words [click, verify, account, expires] were strong indicators of phishing.'

or URL detections: the top-contributing lexical features are listed. Example: 'Flagged because: IP address used as hostname, login keyword found in path, suspicious TLD detected.'

For deepfake detections: for images, the frequency bands that showed GAN upsampling artifacts are identified. For audio, the MFCC and LFCC frames that deviated most from natural speech are highlighted, showing exactly where in the recording the synthetic signal was strongest.

The X-TIS explanation is always presented in plain language next to the risk score. A user without a machine learning background should be able to read it and understand why the system flagged something.

6.11 Alert Generation and Delivery

After X-TIS produces the explanation, a structured alert is assembled containing the threat category, severity level, risk score, and the plain-language X-TIS explanation. As shown in Figure 2, the alert is delivered through one of three channels: Web Dashboard, Browser Extension, or Mobile App.

Through the Web Dashboard, the full alert is displayed. For network threats, the visual subgraph of the involved services is shown alongside the explanation and risk score. The dashboard also maintains a running alert history where teams can filter by category or severity, and mark each alert as reviewed, escalated, or dismissed.

Through the Browser Extension, a risk badge appears on the URL in the browser toolbar as soon as analysis is complete. If the URL is flagged, the badge shows a warning and a tooltip shows the X-TIS reason in context the user does not need to leave the browser.

CHAPTER 7 EXPERIMENTATION

7.1 Part 1 Experimentation

7.1.1 Overview

This project implements an end-to-end hybrid threat intelligence system that combines:

- Content security (Text + URL)
- Network-flow risk scoring (Transformer)
- Service-level graph intelligence (GNN)
- Unknown/zero-day anomaly detection (Autoencoder)
- X-TIS style explainability (feature attribution + subgraph context)
- Analyst-facing reporting (HTML website + per-module reports)

The complete threat coverage is organized into Categories A–H:

- **A.** A. Spam, Phishing & Scam Attacks (message/email content)
- **B.** B. Malicious URL & Website Attacks (URL strings + optional redirect/HTML signals)
- **C.** C. Network-Based Attacks (scan/recon/brute-force style indicators)
- **D.** D. Lateral Movement & Internal Attacks (SMB/NetBIOS style movement proxies)
- **E–H.** E–H. Advanced network threat taxonomy (C2, exfiltration, malware behavior, multi-stage/unknown)

In experimentation, the pipeline is treated as a set of interoperable stages:

- Ingest telemetry (offline datasets or security-stack logs)
- Normalize into flow/message/URL records
- Run detectors per modality (Transformer/GNN/Autoencoder + heuristic baselines)
- Attach taxonomy labels (A–H) and X-TIS explanations
- Export alerts and render reports (CSV + HTML website)

All artifacts referenced in this chapter are either generated by the project scripts or bundled under deliverables/final_bundle/report_assets/.

7.1.2 Data Preprocessing Pipeline

Flow-Level Extraction (KDD/NSL-KDD style records)

Raw network-flow style data is consumed from KDD-format files and converted into a pandas-friendly CSV (kdd_preprocessed.csv). Each record includes fields such as:

- service (categorical)
- protocol_type (TCP/UDP/ICMP categorical)
- src_bytes, dst_bytes (traffic volume)
- duration and several rate/count features available in KDD-derived tables

Label preprocessing follows a strict binary threat definition: `is_attack` = 1 if label $\neq$ normal, else 0.

Optional “Security Stack” Ingestion (Zeek / Suricata / PCAP)

To future-proof the system for real deployments, the repo includes an ingestion layer that converts common SOC telemetry into normalized flow records:

- Zeek logs (conn/dns/http/ssl)
- Suricata eve.json
- PCAP → flows via TShark + CICFlowMeter

This ingestion is implemented under `security_stack/` and outputs a normalized CSV (e.g., `stack_flows.csv`). This pathway is designed so the same downstream detectors can run on either offline datasets or real sensor exports.

Service Identification and Node Creation (for GNN)

Service identification is obtained directly from the dataset’s service field. Each unique service becomes a node in the service graph, and node attributes are computed by aggregating record-level features.

Text Preprocessing (Category A)

For content-based threats, raw SMS/email text is parsed into message records and scored with heuristic sub-detectors (spam/phishing/scam/BEC/spear-phish and malicious-link cues). Inputs include `spam.csv`, `SMSSpamCollection`, and `easy_ham`-style corpora.

URL Extraction and Normalization (Category B)

URLs are extracted from messages (Category A X-TIS JSON) or read from a URL list (urls.txt). URLs are normalized and scored using lexical/structure-based risk signals, with optional redirect and HTML inspection when internet access is available.

Statistical Feature Aggregation (Service-Level for GNN)

Per-service features are computed using group-by aggregation. The “no-leak” evaluation (reported in Chapter 8) intentionally restricts features to the following three aggregates:

- src_bytes_mean
- dst_bytes_mean
- count (number of records aggregated for the service)

This “no-leak” feature set is explicitly listed in report_assets/generated_report/metrics.json and is designed to avoid using label-proxy features during evaluation. Additional enriched aggregates may be present when generated by graphbuilder_clean.py (e.g., duration_mean/std and selected KDD rate/count means), but the reported no-leak metrics are based on the three-feature subset above.

Temporal Feature Extraction (Scope in this project)

KDD/NSL-KDD records do not provide full packet timestamp sequences required for true temporal burst/seasonality modeling. Therefore, this project focuses on service-level statistical aggregation for reproducible evaluation; time-aware features are treated as a future extension for PCAP-derived flow pipelines.

7.1.3 Graph Construction

Edge Definition (Service–Protocol Structure)

The project constructs a bipartite graph between:

- Service nodes (e.g., http, ftp)
- Protocol nodes (e.g., tcp, udp, icmp)

An edge is created whenever a service appears with a given protocol in the dataset. For the GNN, this bipartite structure is converted into a service–service adjacency matrix by connecting two services if they share a protocol neighbor.

Graph Representation

For service-level GNN training, the graph is represented by:

- Adjacency matrix $A \in \mathbb{R}^{(N \times N)}$ derived from the service–service connections
- Feature matrix $X \in \mathbb{R}^{(N \times F)}$ (no-leak experiment uses $F = 3$)
- Label vector $y \in \{0,1\}^N$ where $y = 1$ iff `attack_rate` > 0 for that service

Self-loops are included during GNN normalization by adding the identity matrix prior to computing the normalized adjacency.

7.1.4 Dataset Normalization and Balancing

Feature Scaling

Service-level features are standardized using z-score normalization (`StandardScaler`) fit on the training split only:

$$x' = (x - \mu) / \sigma$$

This prevents train/test leakage and keeps features comparable across magnitude scales.

Class Imbalance Handling

Service labels are imbalanced in typical network datasets. Instead of synthetic oversampling, the GNN training uses:

- Stratified train/validation split
- Class-weighted cross entropy, where class weights are computed from the training split only

7.1.5 Training Configuration (for reproducible runs)

The experiments use multiple models. Where training is required, the project is designed to run on CPU locally or on a GPU machine (then copy back only the artifact outputs).

GNN (service-level)

The no-leak GNN experiment is configured for reproducible execution with:

- Optimizer: Adam
- Early stopping: monitored on validation ROC-AUC
- Default max epochs: 200
- Patience: 25 epochs

These values correspond to the defaults used by `train_eval_gnn_noleak.py`.

Tabular Transformer (flow-level)

The Transformer training pipeline is implemented under `transformer_tabular/` and supports:

- Preprocessing for KDD/CICIDS2017/UNSW-NB15
- FT-Transformer style training
- Evaluation and detection-mode scoring

For the bundled report, detection metrics are computed at `threshold = 0.5` and summarized in `report_assets/generated_report/metrics.json`.

Autoencoder (anomaly)

The anomaly detector is implemented under `anomaly_autoencoder_unsw/` and trains an autoencoder on NORMAL traffic only. It produces:

- `test_scored.csv` with `anomaly_score` per record
- `threshold.json` (e.g., `p99_5` percentile)
- `metrics.json` for anomaly classification

7.1.6 Output

The final output of this stage includes the artifacts required for downstream models and for operational reporting:

- Preprocessed dataset: `kdd_preprocessed.csv`
- Service statistics: `service_stats.csv` (and optionally `service_stats1.csv` with richer aggregates)
- Graph structure: `service_protocol_graph.gpickle`
- Generated website artifacts: `report_assets/generated_report/`
- Per-module HTML reports (bundled): `report_assets/module_reports/`

7.2 Phase 2: Graph Neural Network Training and Attack Detection

7.2.1 Overview

This phase trains a Graph Neural Network to classify network services as safe (0) or attack-linked (1) using aggregated service features and graph neighborhood context. The core goal is to produce an interpretable service-centric risk ranking that can be operationalized as a monitoring/containment priority list.

7.2.2 GNN Architecture Design

The project uses a lightweight Graph Convolutional Network (GCN-style) model (SimpleGCN), implemented as two linear layers with message passing via a normalized adjacency matrix:

- Layer 1: $\text{Linear}(F \rightarrow H) \rightarrow \hat{A} \text{ aggregation} \rightarrow \text{ReLU}$
- Layer 2: $\text{Linear}(H \rightarrow 2) \rightarrow \hat{A} \text{ aggregation} \rightarrow \text{logits}$
-

where $\hat{A}$ is the symmetrically normalized adjacency with self-loops:

$$\hat{A} = D^{-1/2} (A + I) D^{-1/2}$$

7.2.3 Hyperparameter Strategy

This deliverables bundle reports a fixed, reproducible configuration rather than an exhaustive grid search. The training loop supports tunable parameters (hidden_dim, learning rate, weight decay, epochs, patience), but the reported results correspond to the defaults used during generation of report_assets/generated_report/metrics.json.

7.2.4 Model Training Details

Loss Function

Multi-class cross entropy with class weights (computed from the training split only):

$$\mathcal{L} = -(1/N) \sum_i \sum_{c \in \{0,1\}} w^c \cdot y_i^c \cdot \log(p_i^c)$$

Regularization and Generalization Controls

- L2 regularization via optimizer weight_decay
- Early stopping on validation ROC-AUC (patience-based)

7.2.5 Hardware and Software Specifications

- Programming language: Python 3.11 (bundle-generated environment)
- Deep learning: PyTorch
- Data processing: pandas, NumPy, scikit-learn
- Graph utilities: NetworkX (graph creation/visualization)
- Device support: CPU by default, CUDA acceleration if available

7.2.6 Input and Output Structure

Input

- Number of services in the bundled report: $N = 70$
- No-leak feature dimensionality: $F = 3$ (src_bytes_mean, dst_bytes_mean, count)

Output

Per service node:

- Predicted probability (threat_score) and binary prediction (is_malicious)
- Alert level derived from threat_score thresholds (LOW/MEDIUM/HIGH/CRITICAL)

Artifacts produced by the detection and evaluation pipeline are captured in the generated report folder (see Chapter 8).

7.2.7 Evaluation Metrics

The evaluation reports standard binary classification metrics:

- Precision: $TP / (TP + FP)$
- Recall: $TP / (TP + FN)$
- F1-score: $2 \times (Precision \times Recall) / (Precision + Recall)$
- ROC-AUC: area under ROC curve (threshold-robust probability ranking)

In the current deliverables bundle, the stored no-leak evaluation artifact includes accuracy and a confusion summary, and Chapter 8 reports those values.

7.3 Phase 3: Attack Pattern Discovery and Threat Categorization

7.3.1 Threat Categorization (E–H)

For operational reporting, GNN-flagged services are mapped into four higher-level category groups for analyst readability:

- **E.** E. Botnets & Command-and-Control (C2)
- **F.** F. Data Exfiltration Attacks
- **G.** G. Malware Behavioral Attacks
- **H.** H. Multi-Stage / Unknown Attacks

Categorization is applied during detection-mode reporting, and the bundle exports per-category HTML pages (see Section 8.11).

7.3.2 Risk Ranking Output

The primary risk score used for ranking is the GNN threat_score (probability-like output). The operational report provides:

- A sorted list of services by threat_score
- Counts by alert level (LOW/MEDIUM/HIGH/CRITICAL)
- E–H categorization summaries for flagged services

7.4 Phase 4: Flow-Level Transformer Training and Attack Detection

7.4.1 Overview

This phase provides flow/record-level attack probability scoring using a tabular Transformer (FT-Transformer style). The model outputs a probability-like score $P(\text{attack})$ per row and supports detection-mode exports that are later aggregated for reporting and (optionally) for service-level fusion.

7.4.2 Inputs and Preprocessing

The Transformer supports multiple datasets (KDD, CICIDS2017, UNSW-NB15). Preprocessing scripts normalize categorical vocabularies and numeric scaling and write processed datasets under transformer_tabular/data/processed/.

7.4.3 Output

The detection pipeline exports per-row detections to CSV and produces a risk histogram used in the consolidated website report.

7.5 Phase 5: Anomaly Detection (Unknown / Zero-Day)

7.5.1 Overview

This phase detects unknown or out-of-distribution behavior using an autoencoder trained on normal-only traffic. The anomaly score is reconstruction error (MSE), and a percentile threshold (e.g., p99.5) converts the score into a binary anomaly flag.

7.5.2 Output

The scored test set and threshold artifacts are exported and included in the consolidated report bundle for review.

7.6 Phase 6: Content Threat Detection (Text + URL)

7.6.1 Category A: Text Threats (Spam / Phishing / Scam)

The `text_threats/` module processes message content (SMS/email text) and produces:

- A detections CSV with per-message risk and subtypes
- X-TIS style JSON/CSV explanations for top signals (urgency language, suspicious tokens, etc.)
- A single-file HTML report for analyst review

7.6.2 Category B: URL Threats (Malicious URLs / Websites)

The `url_threats/` module processes URL strings and produces:

- Per-URL risk scores and best-effort subtype labels
- X-TIS style explanations (top lexical/structure features)
- A single-file HTML report for analyst review

When deployed in a security stack, the flagged URLs can be exported as a blocklist candidate for use in a proxy/DNS filtering system.

7.7 Phase 7: Reporting Website, Alerts, and Analyst Workflow

7.7.1 Analyst Website (Implemented)

The project generates a self-contained HTML website that consolidates metrics, plots, and the E–H detection pages. This is the primary artifact for reviewers and analysts.

How to run and view alerts (implemented in this bundle):

- Install Python dependencies (project root environment).
- Run the report generator: `deliverables/final_bundle/generate_project_report.py`
- Open the website entry point:
`deliverables/final_bundle/report_assets/generated_report/index.html`

Intellectual Smart Hybrid AI System Real-Time Threat Intelligence in the Internet

The report folder also contains CSV exports (per-model and per-module detections) to support downstream enforcement tooling.

7.7.2 Alerts and Enforcement (Implemented vs Planned)

Implemented:

- Detection scripts output CSV alerts and a dashboard visualization.
- The alert report includes recommended actions (e.g., isolate a critical service, review firewall rules).

Planned/Integration:

- Automatic blocking (firewall/proxy/DNS) can be integrated by consuming the exported flagged services/URLs.
- “Silent mode” (do not show end-user popups) is supported as an operational concept: run detectors in the background and expose results only in the analyst dashboard.

7.7.3 Extension Integration (Planned)

The project roadmap includes surfacing the same report outputs inside an extension-style UI (e.g., a desktop or IDE panel) for easier triage. In this bundle, the implemented UI is the HTML reporting website and the single-file module reports.

7.8 Continuous Learning and Model Updates (Planned)

To reduce long-term risk from drift and new attack variants, the system is designed to support continuous learning through:

- Periodic retraining on newly labeled flows/messages
- Threshold recalibration (especially for the anomaly autoencoder)
- Drift monitoring (distribution shift alerts)
- Human-in-the-loop feedback from analyst decisions

CHAPTER 8 RESULTS

8.1 Network Graph Visualization and Structural Analysis

8.1.1 Service–Protocol Interaction Graph (KDD/NSL-KDD derived)

The service interaction structure is visualized as a service $\leftrightarrow$ protocol graph.

- Nodes represent services and protocol types.
- Edges connect each service to the protocol(s) it appears with in the dataset.
- The bundled experiments include 70 services.

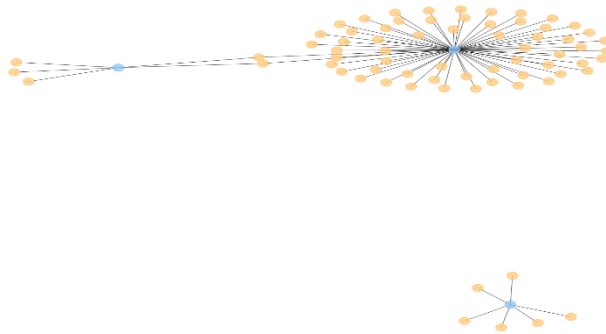


Fig. 8.1: Service–Protocol Interaction Graph (service_protocol_graph.png)

This structural prior is used to build a service–service adjacency matrix (services that share a protocol neighbor are connected), enabling neighborhood-aware GNN inference.

8.1.2 Threat Score “Heatmap” (Operational Dashboard)

Risk is visualized using a detection dashboard that summarizes the distribution of GNN threat scores and the top ranked services.

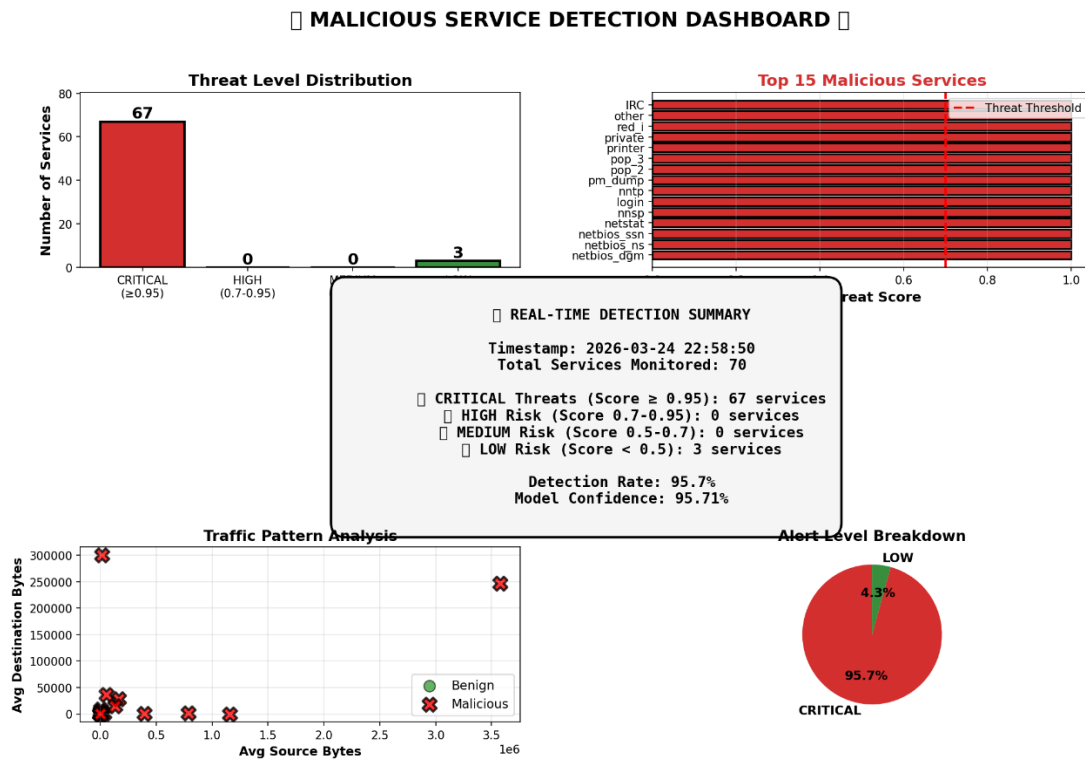


Fig. 8.2: GNN Threat Score Dashboard (gnn_threat_dashboard.png)

Color scheme used in the dashboard:

- **CRITICAL:** CRITICAL: threat_score ≥ 0.95
- **HIGH:** HIGH: $0.70 \leq \text{threat_score} < 0.95$
- **MEDIUM:** MEDIUM: $0.50 \leq \text{threat_score} < 0.70$
- **LOW:** LOW: threat_score < 0.50

8.2 GNN Model Performance and Robustness (Service-Level)

8.2.1 Experimental Setup (No-Leak Evaluation)

To avoid target leakage, the reported evaluation uses only the following aggregated service features:

- src_bytes_mean
- dst_bytes_mean
- count

The binary label is derived at the service level using attack_rate > 0 .

8.2.2 ROC Analysis (No-Leak)

ROC-AUC is reported in the consolidated metrics artifact:

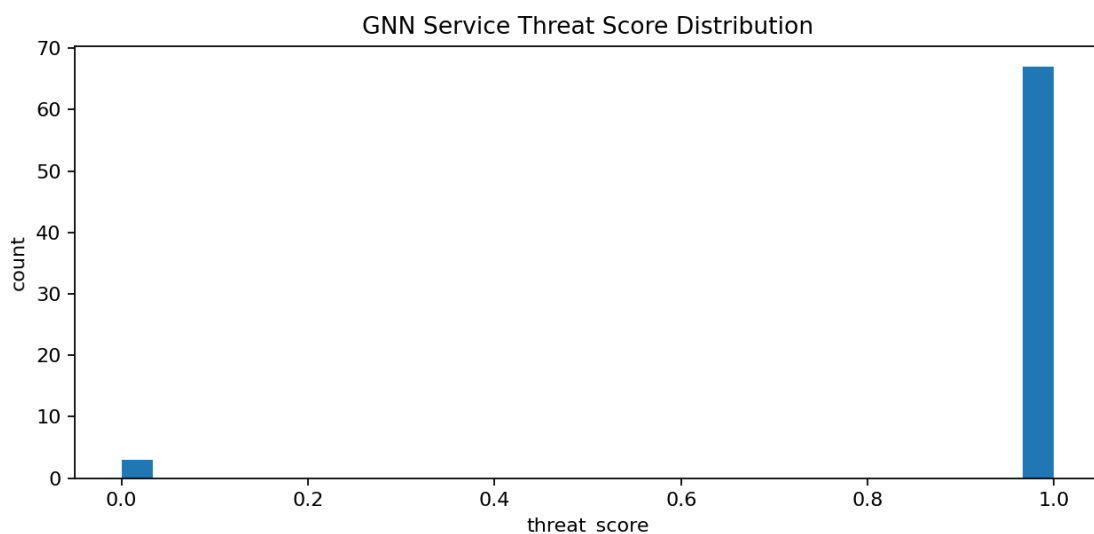
Model	AUC	Accuracy	Precision	Recall	F1	Notes
GNN (service-level)	0.9735	95.71%	97.01%	98.48%	0.9774	No-leak eval. Features: src_bytes_mean, dst_bytes_mean, count. Services: 70. TP=65, TN=2, FP=2, FN=1.
Transformer (flow-level)	0.9998	99.52%	99.33%	99.65%	0.9949	Rows scored: 50,000. Threshold: 0.5.
Autoencoder (anomaly)	0.8652	65.29%	97.78%	50.14%	0.6628	Threshold: p99_5 = 0.053876. Test samples: 175,341. Input dim: 61.

Table.1: ROC Curve – No-Leak GNN Evaluation

8.2.3 Quantitative Results (No-Leak)

The no-leak GNN results stored are:

- Services evaluated: 70
- ROC-AUC: 0.9735
- Accuracy: 0.9571
- Precision: 0.9701
- Recall: 0.9848
- F1-score: 0.9774



8.2.4 Interpretation

The GNN's key advantage is neighborhood-aware message passing: a service's risk estimate is influenced not only by its own aggregated traffic statistics but also by structurally related

services. This matches security reality where suspicious behavior often manifests as correlated signals across a set of related services/protocols.

8.3 Attack Risk Stratification (Service Ranking)

8.3.1 Top-Risk Services

The operational pipeline outputs a ranked list of services by threat_score and flags high-risk services for analyst review.

8.3.2 Practical Security Meaning

Service ranking supports actionable defenses such as:

- Prioritized monitoring (SIEM/IDS correlation)
- Service segmentation and access control tightening
- Incident response triage (focus on the highest-risk services first)

8.4 Fine-Grained Attack Pattern Analysis using X-TIS (Explainability)

X-TIS-style explainability is provided at two levels:

- Feature attribution: highlights which aggregated service features most contribute to a high threat score.
- Subgraph visualization: extracts the local neighborhood around a high-risk service to show how risk may propagate through related services.

8.4.1 Subgraph 1 – IRC-Mediated Command-and-Control (C2)

IRC-like services are frequently abused for botnet control channels. A high-risk IRC node with suspicious neighbors indicates potential C2 coordination and multi-host compromise patterns.

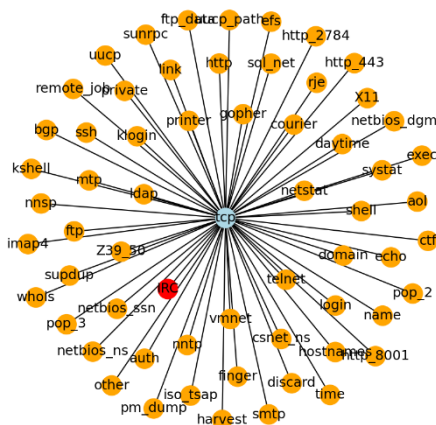


Fig. 8.6: IRC Subgraph – C2 Coordination Pattern (IRC_subgraph.png)

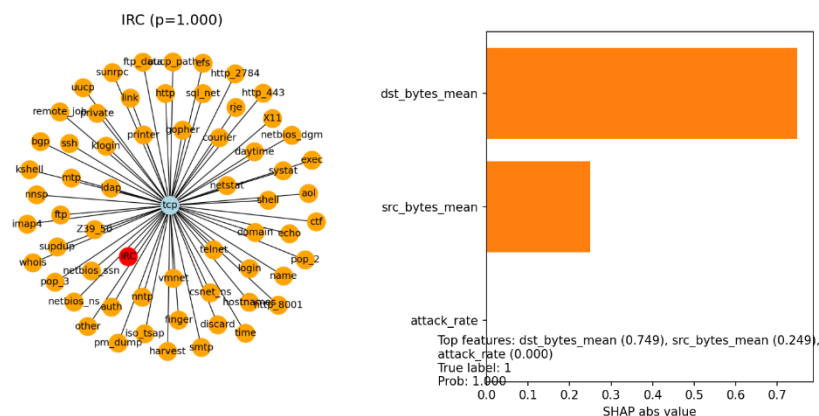


Fig. 8.7: IRC SHAP Feature Attribution (annotated_shap_IRC.png)

8.4.2 Subgraph 2 – NetBIOS/SMB-Style Lateral Movement Cluster

Tight clustering around NetBIOS/SMB-related services is consistent with Windows lateral movement behaviors (host discovery, credential reuse attempts, remote service probing).

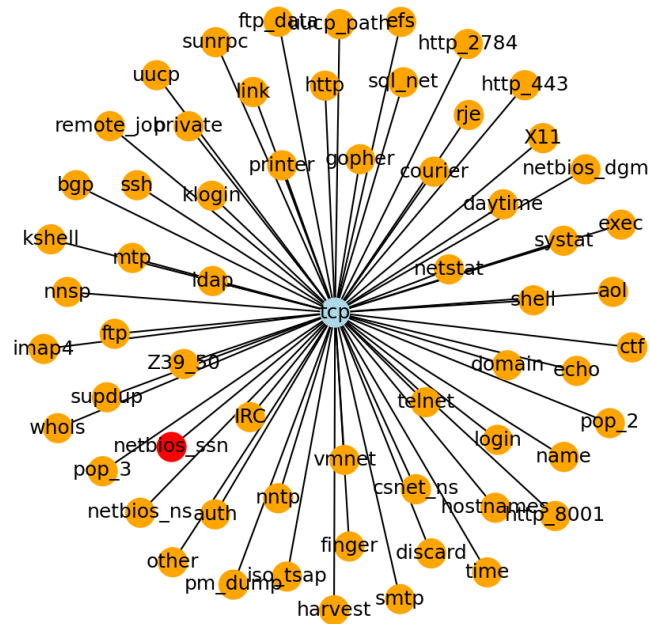


Fig. 8.8: NetBIOS-SSN Subgraph (netbios_ssn_subgraph.png)

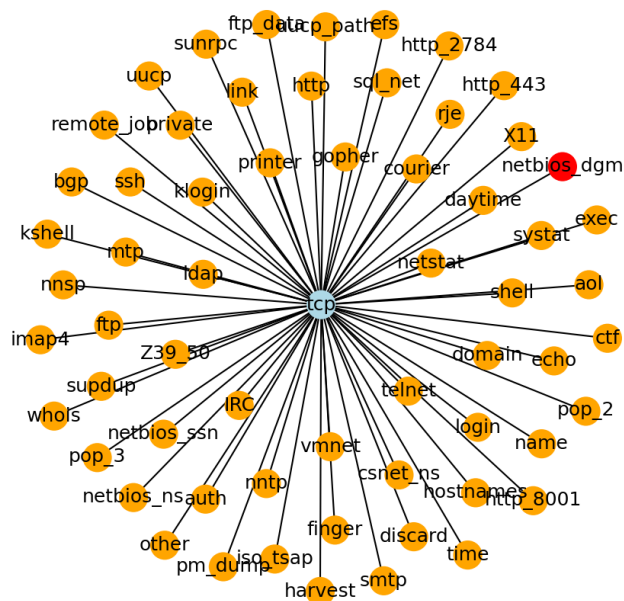


Fig. 8.10: NetBIOS-DGM Subgraph (netbios_dgm_subgraph.png)

8.4.3 Subgraph 3 – Credential Harvesting / Memory Dumping Signals

Process memory dumping is a common technique for extracting secrets and credentials, often preceding mail/service access and eventual exfiltration.

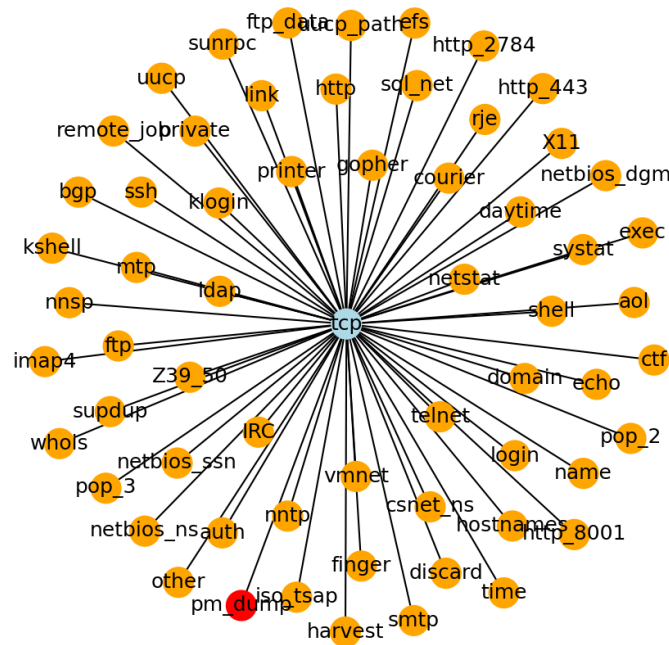


Fig. 8.11: PM_Dump Subgraph (pm_dump_subgraph.png)

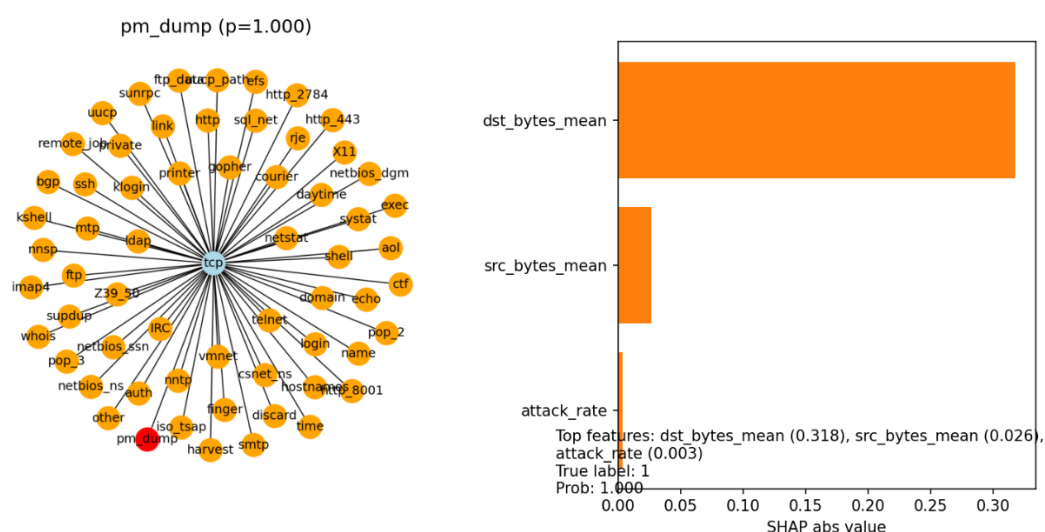


Fig. 8.12: PM Dump SHAP Feature Attribution (annotated shap pm dump.png)

8.5 Detection-Mode Output (Operational Report)

The generated website provides an analyst-friendly view of GNN detections and supporting plots.

Website entry point

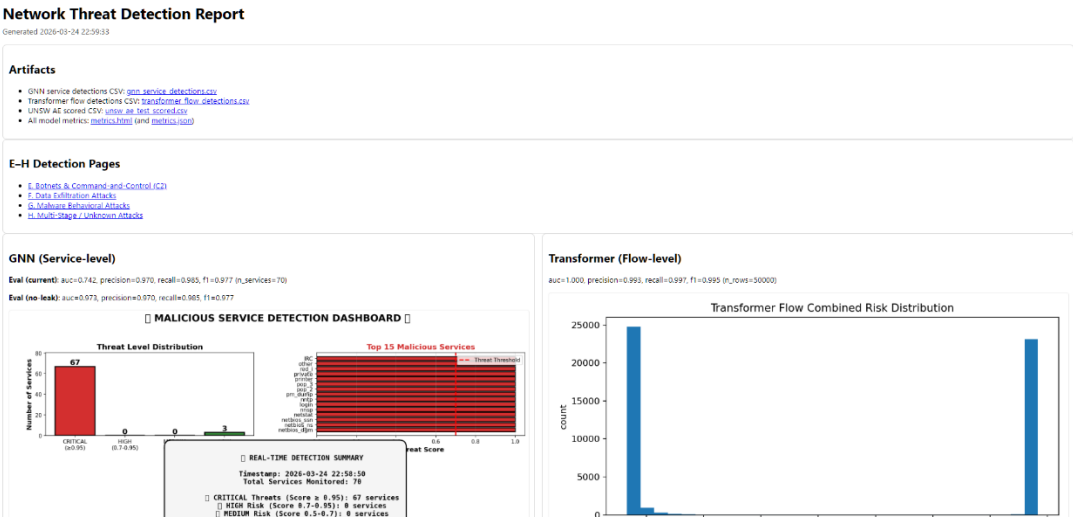


Fig. 8.13: Website Entry Point

Consolidated metrics

Consolidated Metrics (All Models + Project Summary)

Model	AUC	Acc	Prec	Rec	F1	Notes
GNN (service-level)	0.9735	0.9571	0.9701	0.9848	0.9774	No-leak evaluation. Features: src_bytes_mean, dst_bytes_mean, count.
Transformer (flow-level)	0.9998	0.9952	0.9933	0.9965	0.9949	Rows scored: 50000
Autoencoder (anomaly)	0.8652	0.6529	0.9778	0.5014	0.6628	Threshold: p99_5=0.053876

Project-level detection counts (bundled run):
Services monitored: 70
GNN services flagged: 67
Transformer rows scored: 50000
Transformer predicted attack: 23522

E-H totals:

- E. Botnets & Command-and-Control (C2): 3
- F. Data Exfiltration Attacks: 1
- G. Malware Behavioral Attacks: 9
- H. Multi-Stage / Unknown Attacks: 45291

Fig. 8.14: Consolidated metrics

Metrics one-pager

Model & Project Metrics

Generated 2026-03-24 22:59:33 — [Back to main report](#) — [Download JSON](#)

GNN (Service-level)

source	auc	precision	recall	f1	n_services
eval	0.7424	0.9701	0.9848	0.9774	70
eval_noatak	0.9735	0.9701	0.9848	0.9774	70

Transformer (Flow-level)

n_rows	auc	precision	recall	f1	accuracy	threshold
50000	0.999829	0.993283	0.996545	0.994911	0.99522	0.5

UNSW Autoencoder (Anomaly)

artifact_test_auc	artifact_test_precision	artifact_test_recall	artifact_test_f1	threshold_method	threshold_value	recomputed_n_rows	recomputed_auc	recomputed_precision	recomputed_recall	recomputed_f1	recomputed_accuracy
0.865167	0.977775	0.501353	0.662837	p99_5	0.053876	175341	0.865167	0.977775	0.501353	0.662837	0.652854

Project Summary

services_monitored	gnn_services_flagged	transformer_rows_scored	transformer_pred_attack
70	67	50000	23522

Fig. 8.15: Metrics One-Pager Summary (metrics_one_pager.png)

8.6 Transformer Model Results (Flow-Level)

The flow-level Transformer metrics are reported (computed for the bundled run at threshold = 0.5):

- Rows scored: 50,000
- ROC-AUC: 0.9998
- Accuracy: 0.9952
- Precision: 0.9933
- Recall: 0.9965
- F1-score: 0.9949

Primary output:

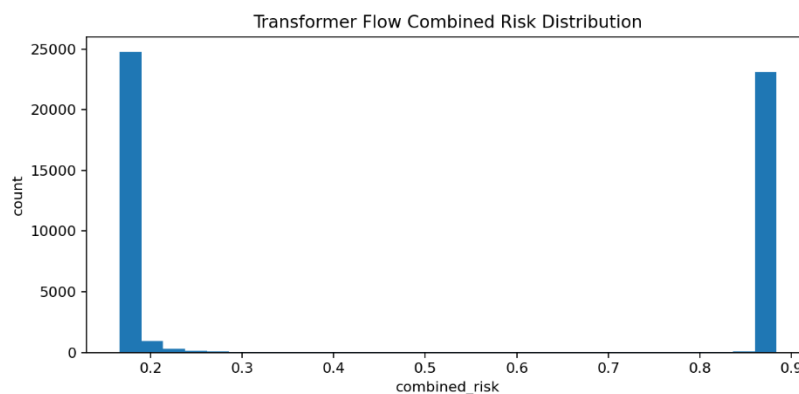


Fig. 8.16: Transformer Combined Risk Histogram (transformer_combined_risk_hist.png)

8.7 Autoencoder Anomaly Results (Unknown / Zero-Day)

The anomaly autoencoder metrics are reported in `report_assets/generated_report/metrics.json` (recomputed on the scored test set):

- Rows scored: 175,341
- ROC-AUC: 0.8652
- Accuracy: 0.6529
- Precision: 0.9778
- Recall: 0.5014
- F1-score: 0.6628
- Threshold method: p99_5 (threshold = 0.053876)

Primary output:

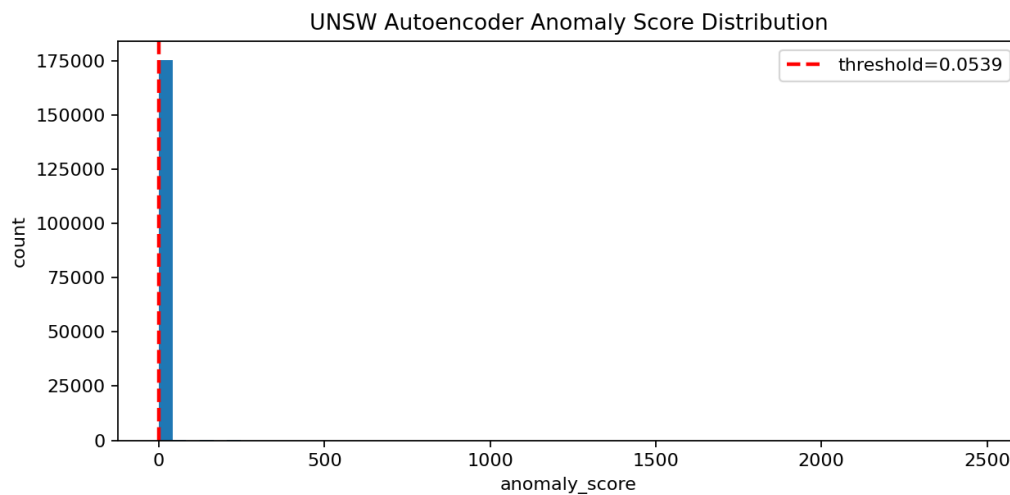


Fig. 8.17: Autoencoder Anomaly Score Histogram (ae_anomaly_score_hist.png)

8.8 Category A Results: Text Threat Detection (Spam / Phishing / Scam)

The text threat detector produces labeled evaluation metrics for standard corpora (heuristic baseline). Bundled run summaries:

spam.csv (labeled):

- Messages: 5,572
- Flagged: 302
- Accuracy / Precision / Recall / F1: 0.9079 / 0.8874 / 0.3588 / 0.5110

SMSSpamCollection (labeled):

- Messages: 5,574
- Flagged: 302
- Accuracy / Precision / Recall / F1: 0.9080 / 0.8874 / 0.3588 / 0.5110

Spam.csv Text Threat Report

Generated from: text_threats/out_spam_csv_detections_full.csv and text_threats/out_spam_xls_full_xls_outputs_text.json

Metrics (labeled subset): accuracy=0.908, precision=0.887, recall=0.359, f1=0.511 (tp=268, tn=4791, fp=34, fn=479)

Overview

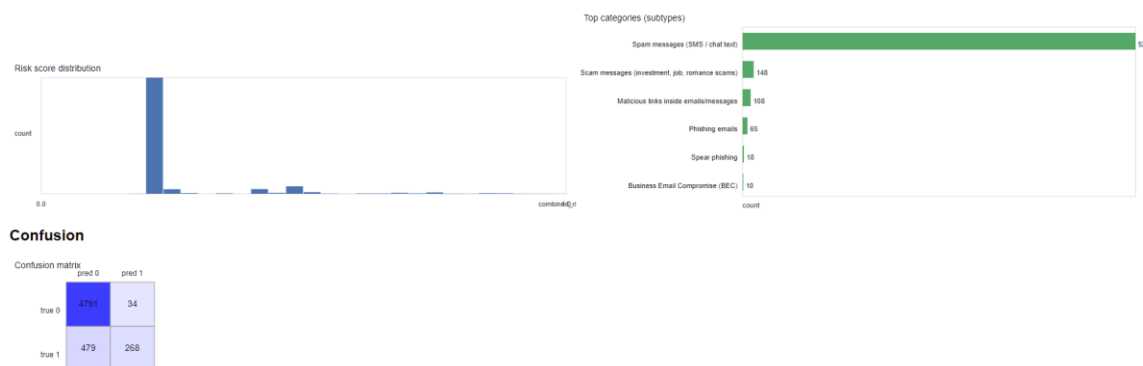


Fig. 8.18: SMS Spam Collection Text Threat Analysis Report

8.9 Category B Results: Malicious URL & Website Attacks

The URL detector was evaluated on URLs extracted from the Category A corpora (lexical baseline). For both extracted URL sets:

- URLs analyzed: 67
- URLs flagged: 24

URL Threat Report (from SMS URLs)

Generated from: url_threats/from_sms_collection_url_detections.csv

Summary: n_urls=67, n_flagged=24

Overview

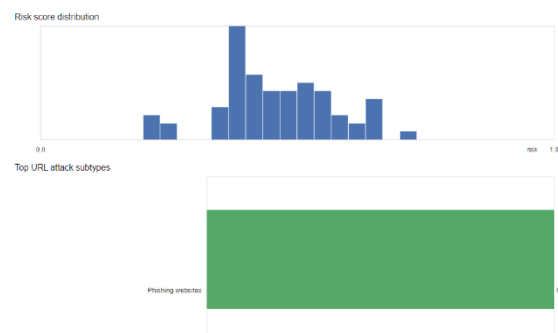


Fig. 8.19: URL Threat Analysis Report – Category B

8.10 Category C/D Results: Network and Lateral-Movement Heuristic Baselines

The network_threats/ module provides best-effort detections for network-based attacks (C) and lateral movement/internal behavior proxies (D) using KDD-like flow fields.

Smoke run summary (bundled):

- Rows processed: 2,000
- Flagged: 51

Network Threat Report (Categories C & D)

Generated from: network_threats/network_cot_detections_smoke_small.csv and network_threats/x_tis_outputs_network/x_tis_outputs_network_smoke_small.json
Summary: n_rows=2000, n_flagged=51

Overview

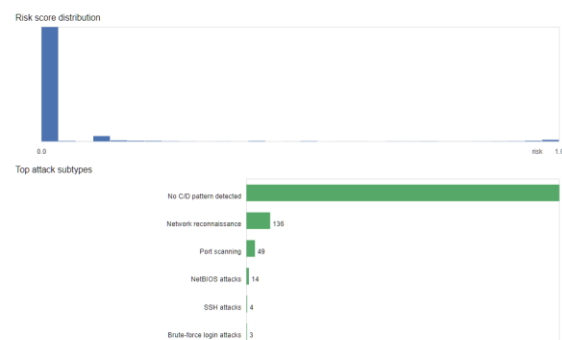


Fig. 8.20: Network C/D Category HTML Report

8.11 Category E–H Threat Detection Results (Outputs + HTML Pages)

8.11.1 E, F, G, H Detection Pages

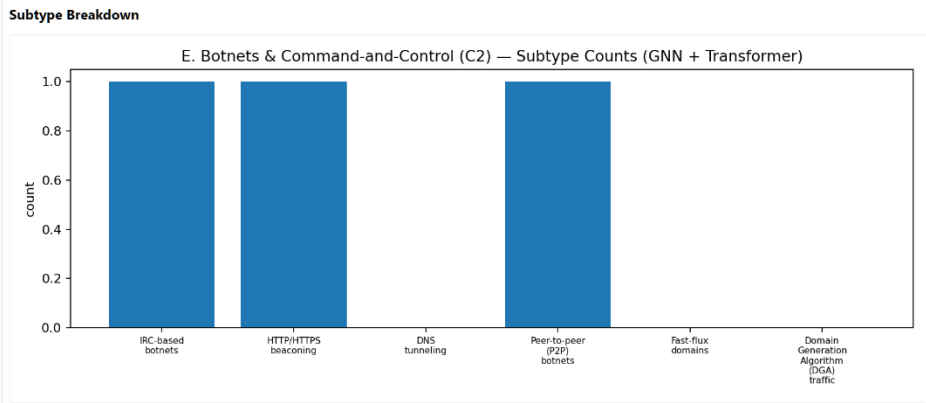
The report exports exactly four category pages:

Fig. 8.21: E, F, G, H Detection Pages

E detections: detections_e-botnets-command-and-control-c2.html

E. Botnets & Command-and-Control (C2)

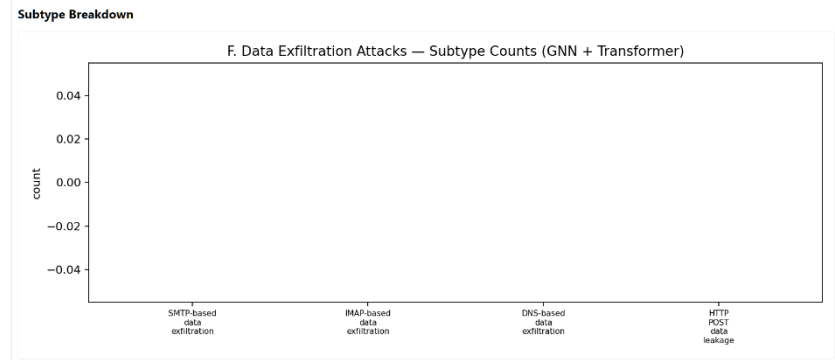
Generated 2026-03-24 22:59:28 — [Back to main report](#)



F detections: detections_f-data-exfiltration-attacks.html

F. Data Exfiltration Attacks

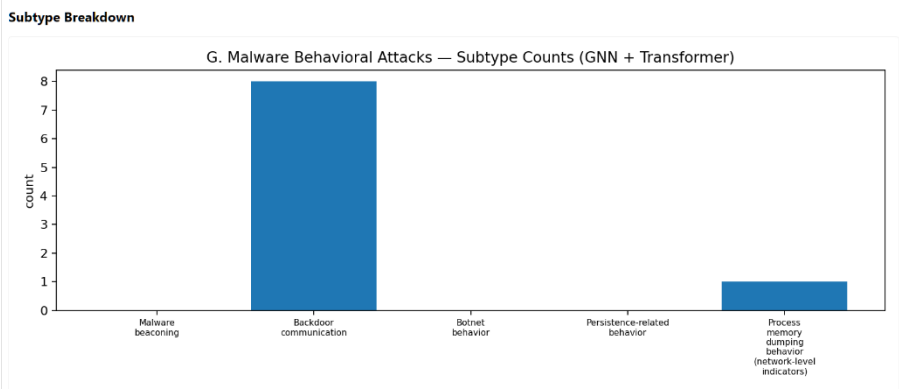
Generated 2026-03-24 22:59:28 — [Back to main report](#)



G detections: detections_g-malware-behavioral-attacks.html

G. Malware Behavioral Attacks

Generated 2026-03-24 22:59:28 — [Back to main report](#)

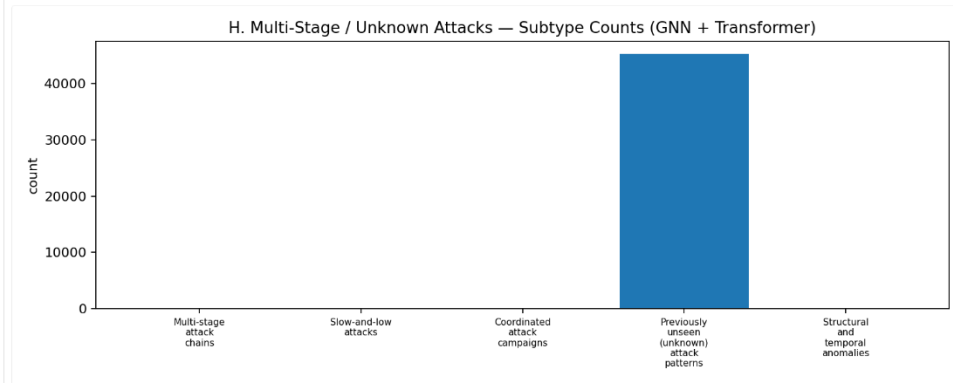


H detections: detections_h-multi-stage-unknown-attacks.html

H. Multi-Stage / Unknown Attacks

Generated 2026-03-24 22:59:28 — [Back to main report](#)

Subtype Breakdown



8.11.2 E–H Category Distribution (GNN portion)

The consolidated metrics artifact includes GNN-only category totals for the bundled run:

- **E total (GNN): 3** E total (GNN): 3 – Botnets & Command-and-Control (C2)
- **F total (GNN): 1** F total (GNN): 1 – Data Exfiltration Attacks
- **G total (GNN): 9** G total (GNN): 9 – Malware Behavioral Attacks
- **H total (GNN): 57** H total (GNN): 57 – Multi-Stage / Unknown Attacks

Supporting subtype count plots (generated as part of report assets):

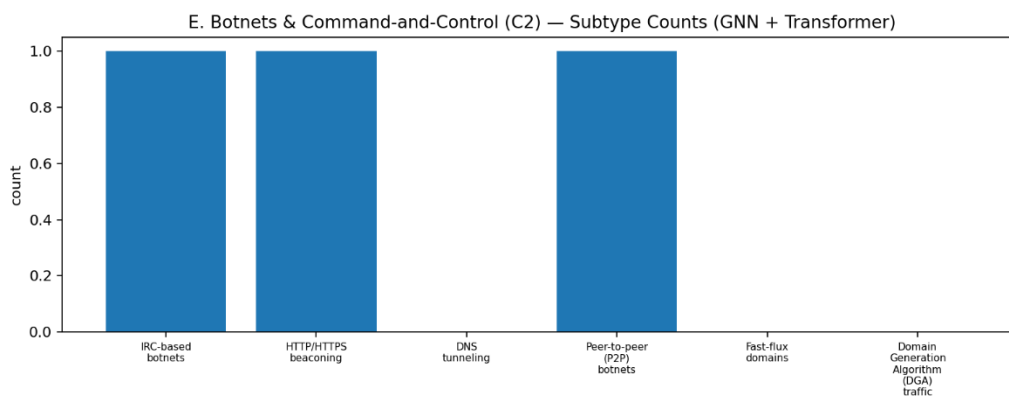


Fig. 8.22: Category E – Botnets/C2 Subtype Counts

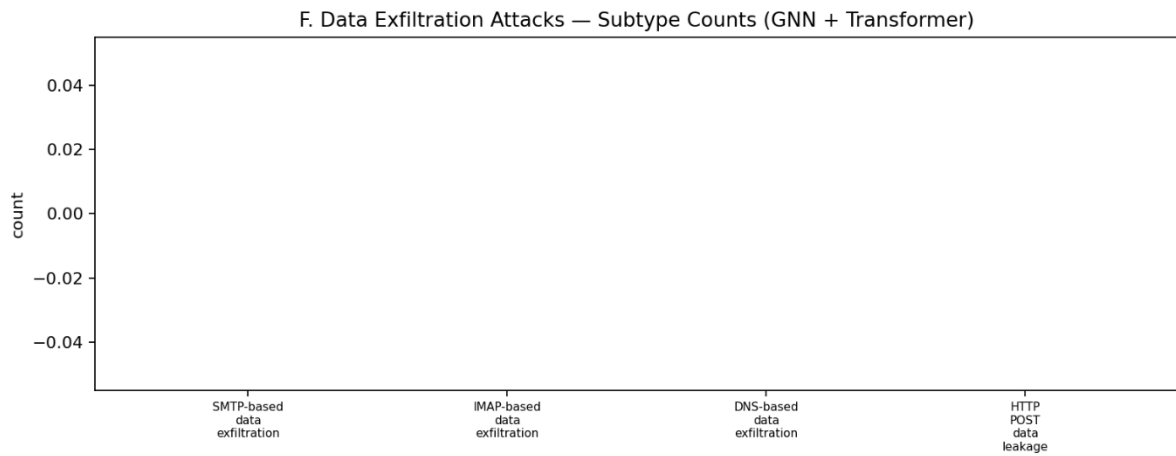


Fig. 8.23: Category F – Data Exfiltration Subtype Counts

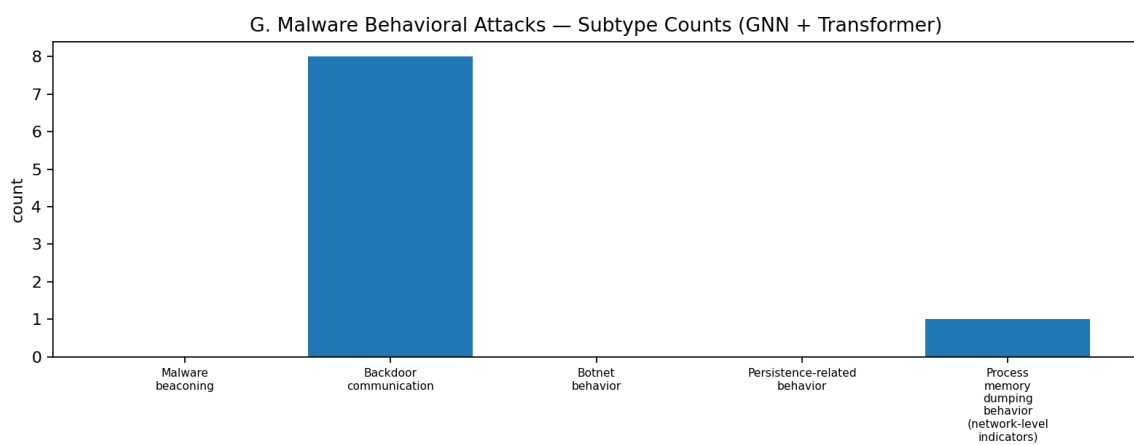


Fig. 8.24: Category G – Malware Behavioral Subtype Counts

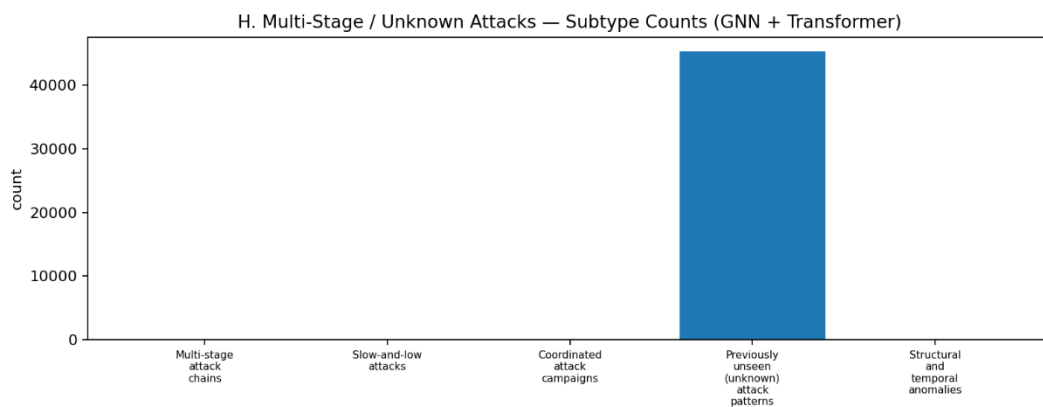


Fig. 8.25: Category H – Multi-Stage/Unknown Subtype

CHAPTER 9 DELIVERABLES

9.1 Complete Hybrid Cyber Threat Detection System

The IS-HAITI system is delivered as a fully functional hybrid AI-based cyber threat detection and intelligence platform capable of detecting a wide range of cyber threats in real-time. The system integrates multiple detection techniques to ensure comprehensive coverage across different attack surfaces.

The implemented system supports detection of:

- Spam, phishing, scam messages, and Business Email Compromise (BEC)
- Malicious URLs, phishing websites, and credential harvesting links
- Network-based attacks such as port scanning, brute-force attacks, SSH/RDP attacks, and reconnaissance
- Lateral movement and internal attacks including SMB and NetBIOS behavior
- Botnet communication and Command-and-Control (C2) traffic
- Data exfiltration activities using DNS, HTTP, SMTP, and IMAP channels
- Malware behavioral patterns such as backdoor communication and persistence activity
- Multi-stage and unknown attacks using anomaly detection and structural analysis

These capabilities are implemented using a combination of Graph Neural Networks, Transformer models, rule-based detection, and explainability mechanisms, ensuring both high accuracy and practical usability.

9.2 Hybrid AI Detection Engine

The core deliverable of the system is a multi-layered hybrid detection engine that combines different AI and rule-based approaches:

- Transformer Model – Performs flow-level analysis to detect attack patterns in network traffic

Intellectual Smart Hybrid AI System Real-Time Threat Intelligence in the Internet

- Graph Neural Network (GNN) – Identifies coordinated and multi-stage attacks by analyzing relationships between services
- Rule-Based Engine – Detects known attack signatures and predefined threat patterns
- Anomaly Detection Module – Uses autoencoder-based techniques to detect unknown and zero-day attacks
- X-TIS (Explainability Module) – Provides clear explanations for every detection using SHAP and LIME

This hybrid approach enables:

- Detection of both known and unknown threats
- Reduced false positives
- Improved contextual understanding of attacks

9.3 Cybersecurity Stack and Practical Implementation

The IS-HAITI system is implemented using a combination of machine learning models and real-world cybersecurity tools, making it suitable for both research and practical deployment.

Network Traffic Processing

- Wireshark / TShark – Used for capturing live network packets
- CICFlowMeter – Converts raw PCAP data into structured flow-based features

Captured traffic is processed into flow data and passed into the detection pipeline for analysis.

Intrusion Detection and Prevention

- The system functions similar to an Intrusion Detection System (IDS) and also as an Intrusion Prevention System (IPS)
- Detects threats in real-time and generates alerts
- Browser extension blocks or warns users about malicious URLs

Threat Detection and Correlation

- Combines AI models with rule-based logic (similar to signature-based systems like Snort)
- Enables detection of complex, multi-stage, and coordinated attacks

Explainability and Analysis

- Uses SHAP and LIME within X-TIS to explain detections
- Provides feature-level and context-level explanations for alerts

9.4 Continuous Learning and Adaptive Intelligence (CADA)

The system includes a continuous learning mechanism to handle evolving cyber threats:

- Detects changes in incoming data (concept drift)
- Updates models incrementally without full retraining
- Learns from new attack patterns over time
- Improves detection performance continuously

This ensures long-term adaptability and robustness of the system.

9.5 User-Facing Applications

The IS-HAITI system is delivered through multiple user interfaces:

Web Dashboard

- Displays real-time alerts and threat intelligence
- Shows risk scores, attack categories, and explanations
- Provides visualizations including graph-based attack patterns
- Maintains alert history for analysis

Browser Extension

- Monitors URLs during browsing
- Detects malicious links instantly

Intellectual Smart Hybrid AI System Real-Time Threat Intelligence in the Internet

- Displays warnings and risk scores
- Can block access to harmful websites

Mobile Interface

- Sends real-time notifications for critical threats
- Enables remote monitoring

9.6 Automated Threat Response

The system supports proactive security actions:

- Generates alerts for detected threats
- Warns users before accessing malicious content
- Blocks malicious URLs at browser level
- Future enhancement: automatic blocking of threats before user interaction

9.7 Analyst-Friendly Reports and Outputs

The system produces structured outputs for both machines and users:

- CSV / JSON outputs for programmatic use
- HTML reports for human-readable analysis
- Threat classification with severity levels
- X-TIS explanations for each detection

These outputs assist:

- Security analysts
- Researchers
- SOC teams

in understanding and responding to threats efficiently.

9.8 Implementation Artifacts

The project delivers the following artifacts:

- Trained models (Transformer, GNN, Anomaly Detection)
- Detection pipelines for network, text, and URL analysis
- Explainability outputs (SHAP, LIME, subgraph visualizations)
- Evaluation metrics and performance reports

9.9 System Capabilities Summary

The final system provides:

- Real-time threat detection across multiple domains
- Multi-input support (network traffic, text, URLs)
- Explainable AI-based decision making
- Continuous learning and adaptability
- Scalable deployment through APIs and dashboards

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [2] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 18932–18943, 2021.
- [3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.
- [4] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 1024–1034, 2017.
- [5] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pp. 229–238, Seattle, WA: USENIX, 1999.
- [6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 4765–4774, 2017.
- [8] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pp. 1135–1144, San Francisco, CA, 2016.
- [9] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, "Contributions to the study of Department of Computer Science & Engineering, SOE, DSU

Intellectual Smart Hybrid AI System Real-Time Threat Intelligence in the Internet
SMS spam filtering: New collection and results," in *Proceedings of the 2011 ACM Symposium on Document Engineering (DocEng '11)*, pp. 259–262, Mountain View, CA: ACM, 2011.

[10] M. A. Feroz and S. Mengel, "Phishing URL detection using URL ranking," in *Proceedings of the 2015 IEEE International Congress on Big Data*, pp. 635–638, Santa Clara, CA: IEEE, 2015.

[11] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1–11, Seoul: IEEE, 2019.

[12] M. Todisco, X. Wang, V. Vestman, M. Sahidullah, H. Delgado, A. Nautsch, J. Yamagishi, N. Evans, T. Kinnunen, and K. A. Lee, "ASVspoof 2019: Future horizons in spoofed and fake audio detection," in *Proceedings of Interspeech 2019*, pp. 1008–1012, Graz: ISCA, 2019.

[13] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[14] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proceedings of the Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6, Canberra: IEEE, 2015.

[15] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, Madeira, Portugal, 2018.

[16] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pp. 1–6, Ottawa: IEEE, 2009.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, Las Vegas: IEEE, 2016.